

-->

PhyloODB User Manual

PhyloODB is a database-centred system for assembling phylogenomic datasets from publicly available genome assemblies or locally imported fasta proteomes/genomes. It is designed for users who want more flexibility than a one-off marker extraction. Instead of treating dataset construction as a workflow of scripts.

PhyloODB stores assemblies, taxonomic information, BUSCO results, derived libraries, filtering decisions, task history, and export-ready results in a persistent SQLite database. The result is a working environment in which taxon selection, orthology-aware filtering, contamination screening, and dataset export can be repeated and revised without having to rediscover every intermediate step.

This manual is written in the style of a practical user handbook. It begins with the ideas behind the program, then explains how the command-line interface is organised, how tasks are combined, and how a typical project proceeds from metadata acquisition to final export.

Use this manual for: the comprehensive guide. If you want a shorter first pass, start with [Quick Start](#). If you need exact syntax or a compact task list, use the [Command Reference](#). If you want a worked end-to-end example, use the [Tutorial](#).

Contents

- 1. What PhyloODB is for
- 2. The basic model
- 3. Top-level command structure
 - 3.1 Inspection commands
 - 3.2 Execution commands
 - 3.3 Database-control commands
 - 3.4 Storage roots and active working directories
- 4. Creating a database
- 5. Selectors and accession resolution
 - 5.1 Declarative taxon sampling
 - 5.2 Stored accession sets as variables
 - 5.3 Selector presets
 - 5.4 Exclusions and negative selection
 - 5.5 `list assemblies` : metadata, BUSCO columns, filtering, and output modes
 - 5.5.1 Selecting which BUSCO run to use
 - 5.5.2 Metadata
 - 5.5.3 Filtering Assemblies
 - 5.5.4 Sorting Assemblies
 - 5.6 Listing BUSCO runs and Gene Families
 - 5.7 Proteome profiles in listing and selection
- 6. Variables and System Defaults
 - 6.1 Variables and manual configuration
 - 6.2 Default proteome profiles
 - 6.3 Manual BUSCO primary overrides
- 7. Queueing, running, and the daemon model
 - 7.1 Runtime thread defaults
 - 7.2 Scheduling and dependencies
 - 7.3 The daemon and queue inspection
- 8. PhyloODB Tasks

- 8.1 Data acquisition and registration
 - 8.1.1 Taxonomy and assembly metadata
 - 8.1.2 Downloading assemblies
 - 8.1.3 Automatic proteome preparation after download
 - 8.1.4 Local import integrity
 - 8.1.5 Local import and automatic isoform cleaning
 - 8.1.6 The verify tasks
- 8.2 BUSCO and orthology preparation
 - 8.2.1 BUSCO lineage libraries
 - 8.2.2 Running BUSCO on assemblies
 - 8.2.3 BUSCO input modes and proteome profiles
 - 8.2.4 OrthoFinder runs
 - 8.2.5 Derived library construction
 - 8.2.6 Custom library import
- 8.3 Hidden paralog filtering
- 8.4 Decontamination
- 8.5 Export and reporting
 - 8.5.1 `export` : `--require` , `--header` , and related settings
 - 8.5.2 What export writes
 - 8.5.3 Export-aligned BUSCO tree building
- 8.6 Storage and diagnostic helper tasks
- 9. How tasks combine in practice to produce a dataset
- 10. Suggested workflows
 - 10.1 Exploratory survey of a clade
 - 10.2 Building a conservative core library
 - 10.3 Large target sampling after library construction
- 11. Interpreting outputs
- 12. Database management and recovery
 - 12.1 Storage roots: what they are and how they work
 - 12.2 Active and inactive roots in real work
 - 12.3 Moving a whole root versus moving selected data
 - 12.4 What verification happens after moves

- [12.5 Practical SSD/HDD scenarios](#)
- [12.6 The artifact system in depth](#)
- [12.7 Purging records, files, settings, and roots](#)
- [12.8 Rediscovery and filesystem reconciliation](#)
- [12.9 Rediscovery versus verify](#)
- [12.10 Recommended operator discipline](#)
- [13. Practical advice](#)
- [14. Further reading in this repository](#)

1. What PhyloODB is for

PhyloODB addresses a central problem in phylogenomics: how do we identify a set of dependable loci for our study taxa that maximises breadth while ensuring reliable orthology. Several operations are routinely required in such work.

- Assemblies must be discovered and ranked by completeness and quality.
- Candidate taxa must be sampled from the assembly bank.
- Orthology information must be obtained, usually from BUSCO or from a more explicit orthology method.
- Hidden paralogs and suspect sequences must be screened out.
- Exports must be generated in a way that reflects the current state of filtering and organised in a coherent way.

PhyloODB treats all these as tasks in relation to a central database of assemblies. The database records what has been downloaded, analysed, accepted, rejected, and then exported. This design has several consequences.

- The same database can support repeated analyses of a clade without repeating all steps linearly.
- Selectors can be declarative. Instead of manually curating long accession lists, one can ask for, for example, one assembly per genus within a clade, or two assemblies per family, per order within a broader sample.
- Filtering steps are user defined. Paralog filtering and decontamination are not informal post-processing but named tasks with run information tracked.
- The queue and daemon model allows a project to be built incrementally, with tasks depending on one another rather than all logic being embedded in one shell script.
- A single database could hold the resources for multiple projects, and in general the larger the database is the easier it is to produce genomic datasets quickly.

2. The basic model

A PhyloODB project is anchored to a SQLite database file. All commands begin by naming that database.

```
phyloODB my_project.db <command> ...
```

Note: a new database file is not required for each project. A central database can serve several projects and it is recommended to work within the same database to avoid duplication

3. Top-level command structure

The top-level interface is:

```
phyloODB <database>
{list,watch,tree,storage,discover,count,assemblies,selector,queue,status,
run,set,info,create,migrate,clear,purge,reset,kill,cancel}
```

These commands fall into four broad groups.

3.1 Inspection commands

- `list` lists tasks, queue/errors, assemblies, libraries, variables, ranks, or metadata as tables or tsv outputs.
- `list queue --watch` and `list errors --watch` are the canonical live monitors.
The top-level `watch` command is an exact convenience alias.
- `assemblies` is a short-cut for `list assemblies` as this is the most frequent command.
- `selector` saves, inspects, previews, resolves, and deletes named selector presets.
- `count` reports counts for selector-defined assembly sets.
- `tree` renders a selector-defined taxonomic tree and can write a Newick file.
- `info` describes tasks, commands, or the database itself.

3.2 Execution commands

- `queue` queues a task for later execution by the daemon.
- `status` checks a queued task by id or selector and returns script-friendly exit codes.
- `run` follows one task chain in the foreground.

The same task catalogue is available through `queue` and `run`. `queue` submits work to the shared daemon. `run` queues one root task, starts a temporary foreground daemon for that task tree only, and follows that chain through suspension and subtasks until it finishes.

3.3 Database-control commands

- `create` creates and initialises a new database.
- `discover` scans registered genomes roots and can register or rebind assemblies and BUSCO runs already present on disk.
- `set` sets database environment variables.
- `storage` manages storage roots and storage moves.
- `clear`, `purge`, `reset`, `kill`, and `cancel` control task or data state.

These commands are powerful and should be used deliberately. They are not part of routine tutorial usage unless one is explicitly resetting or cleaning a project.

Note: You should never need to delete data from the database. If you wish to change taxa/busco results or libraries you should use variables to control this.

3.4 Storage roots and active working directories

PhyloODB stores filesystem base paths as registered storage roots rather than relying only on environment variables. The practical effect is that the program can know where genomes, libraries, OrthoFinder outputs, exports, reports, logs, and miscellaneous derived artifacts live, even when they are spread across more than one drive. By default these roots are initiated in the folder the database is created.

For `genomes`, `libraries`, `orthofinder`, `exports`, and `logs`, the model is intentionally strict:

- only one root of that kind is active at a time
- the active root is the write target for new data
- inactive roots remain valid locations for already-bound data

This is designed for queue-based working patterns such as:

1. keep an SSD root active for new downloads and active work
2. add a larger HDD root
3. move older genomes or libraries to the HDD
4. continue using those moved data in analyses while keeping new writes on the SSD

If desired, the active root can later be switched explicitly.

Typical commands are:


```
phyloODB my_project.db list roots
phyloODB my_project.db storage add-root --kind genomes --base-path
/mnt/hdd/genomes --label HDD
phyloODB my_project.db storage rename-root HDD --label HDD_ARCHIVE
phyloODB my_project.db storage move-genomes --accessions @OLD_SET --to-
root 7 --apply
phyloODB my_project.db storage activate-root 7
```

Important operational rules:

- `storage add-root` resolves and creates the directory, then verifies that it is readable and writable before registering it
- `storage add-root` creates a non-first strict root inactive by default
- `storage rename-root <id-or-label> --label <new-label>` changes only the unique human-readable label; paths, bindings, and active state are unchanged
- `storage activate-root <id>` makes that root the sole active write target for its kind
- `storage deactivate-root <id>` is allowed, but if no active root remains for that kind then new write-producing work for that kind is suspended until a root is activated again
- `purge roots` deletes root definitions, but bound roots are blocked from deletion in normal use
- `logs` is strict because there should be one current log write target; activate a different logs root to move future log files

The `reports`, `cache`, and `misc` root kinds are different. They are shared operational buckets rather than primary working-root toggles: reports hold task reports, cache holds reusable intermediate files such as BLAST databases, and misc is for auxiliary derived artifacts that do not naturally belong under the main genome/library/orthofinder/export roots.

[Figure placeholder: command tree showing top-level commands and the split between inspection, execution, and control commands.]

4. Creating a database

A project normally begins with:

```
phyloODB my_project.db create --email your_email@domain.com --api-key  
your_NCBI_api_key
```

The `create` command can also accept an explicit taxdump path or working directory. The database then serves as the permanent record for that project.

Once the database exists, the next conceptual stage is to populate it with assembly metadata. In many projects this is the first genuine scientific step, because taxon selection depends on what assemblies are available.

The relevant task is `update-assembly` (aliases `add`, `update-assembly-info`). This task fetches assembly metadata either for a taxid or for explicit accessions. It does not download sequence data. Rather, it registers what is available and records assembly attributes that later selectors can use.

Typical examples are:

```
phyloODB my_project.db queue add --clade Primates  
phyloODB my_project.db queue add --accessions  
GCF_000001515.8,GCF_000151905.2
```

The first form is often the most useful because it populates a broad clade and then lets the user refine sampling with selectors.

TIP: In practice so long as you have a stable internet connection and set the NCBI API key you can update the database with all knowledge immediately.

```
phyloODB my_project.db set var NCBI_API_KEY YOUR_KEY  
phyloODB my_project.db queue add --clade Metazoa
```

This can then be run periodically to update the database with new assemblies.

5. Selectors and accession resolution

One of the most important features of PhyloODB is that datasets can be declared rather than typed out by hand. The **selector** system is the main mechanism for this. Selectors allow for a taxonomic rule to be applied to resolve a set of assemblies, or in practice BUSCO gene datasets.

Selectors are used throughout PhyloODB. The same selector language can drive exploratory commands and real analysis tasks:

```
phyloODB my_project.db list assemblies --clade Primates
phyloODB my_project.db tree --clade Primates --rank genus --quantity 1
phyloODB my_project.db queue download --clade Primates --rank genus --
quantity 1
phyloODB my_project.db queue batch-busco --accessions @PRIMATE_REFS --
lineage mammalia_odb12
phyloODB my_project.db queue export-library --accessions @CORE_SET
phyloODB my_project.db queue paralog-removal --accessions @CORE_SET
```

The common selector inputs include:

- `-c` , `--clade <name>`
- `-i` , `--taxid <id>`
- `-a` , `--accessions <list>`
- `-d` , `--downloaded-only`
- `--not-downloaded`
- `--local-only`
- `--not-local`
- `--primary-only`
- `-af` , `--after` and `-bf` , `--before`
- `-rt` , `--root`
- `--level`
- `--busco-complete-min`
- `--busco-single-min`
- `--has-busco-results`
- `--missing-busco-results`

- `-r` , `--ranks`
- `-q` , `--quantities`
- `--sample-strategy`
- `--sample-seed`

BUSCO aware/library selectors also support `-l` , `--library-name` .

These can be used with `list assemblies` , `count assemblies` , and many tasks routed through `queue` and `run` .

5.1 Declarative taxon sampling

The simplest selector asks for all assemblies within a clade. This will include assemblies that are not yet downloaded.

```
phyloODB my_project.db list assemblies -c Primates
```

A more typical phylogenomic use is rule-based sampling.

```
phyloODB my_project.db list assemblies -c Primates -r family -q 1
```

This means “choose one assembly per family within Primates”, with the internal ranking system deciding which assembly is best in each group. This is much more reproducible than manually browsing accession lists.

By default that ranking prefers, in order: BUSCO score when enabled, `origin=refseq` , assembly level, contig N50, newer release date, then accession as a deterministic tie-break. The project variables `SELECTOR_SCORE_ORDER` and `SELECTOR_BUSCO_BUCKETS` control this default scoring behaviour.

Multi-stage subsampling is also possible.

```
phyloODB my_project.db list assemblies -c Primates -r family,genus -q 2,1
```

This can be read as: keep two genera per family, then one assembly per genus. In practical terms it allows one to control breadth and density separately.

TIP: the `--ranks` flag is not only useful in isolation. It can be used in assembly views to show taxonomic information for assemblies beyond species name. Similarly quantities can be issued without providing a rank, this allows you to select the top n within the clade as a whole.

By default, quantity-limited selectors use PhyloODB's quality ranking rules. If you want a random sample from the same eligible pool, use `--sample-strategy random`. Add `--sample-seed` when the random draw needs to be reproducible.

```
phyloODB my_project.db list assemblies \  
  --clade Primates \  
  --rank genus \  
  --quantity 1 \  
  --sample-strategy random \  
  --sample-seed 20260713
```

Use the default `--sample-strategy rank` when you want the best-ranked assembly in each group. Use `random` when the scientific question calls for stochastic sampling from the available assemblies rather than a quality-ranked representative.

How selectors combine

When more than one positive selector is supplied, PhyloODB first builds a single candidate pool and only then applies rank/quantity sampling.

For example:

- `--accessions`, `--taxid`, and `--clade` contribute to the same candidate pool;
- those positive selectors are combined as a union, not as an intersection;
- ordinary filters such as `--filter`, `--downloaded-only`, `--not-local`, and BUSCO thresholds are applied to that combined pool;
- `--rank / --quantity` and `--ranks / --quantities` then sample from whatever remains after that filtering.

So:

```
phyloODB my_project.db list results \  
  --accessions @BILATERIA \  
  --clade Porifera \  
  --ranks phylum,class,order \  
  --quantities 1,1,1 \  
  --downloaded-only
```

should be read as:

1. start with the accessions stored in variable `@BILATERIA` ;
2. add all assemblies resolved from `Porifera` ;
3. filter to the best assembly per order within the pool;
4. filter to the best assembly per class within the remaining pool.
5. Return the best assembly per phylum within the remaining pool.

It does **not** mean "take only the Porifera members of `@BILATERIA`".

This union behaviour is useful in several common situations:

- widen a hand-curated panel with one whole clade, for example "my existing metazoan core set plus all Porifera";
- keep a fixed set of must-have references while also sampling broadly from a larger lineage;
- add newly available representatives from one focal clade without rebuilding an older stored panel from scratch.

Note: taxid works exactly the same as clade. If two clades have the same name e.g. "Ctenophora" we may choose to give an exact taxid.

5.2 Stored accession sets as variables

A resolved selector can be stored in the database as a named accession set.

```
phyloODB my_project.db list assemblies -c "Pan paniscus" -S BONOBO
```

This stored set can later be reused with `@BONOBO` .

```
phyloODB my_project.db list assemblies -a @BONOBO
phyloODB my_project.db queue download -a @BONOBO
```

This is especially useful in long projects, where the same reference panel or target panel is used across several tasks.

Two related output operations are also available when building panels incrementally:

- `-S` , `--store NAME` , or `--save-set NAME` replaces a stored set with the newly resolved accession set.
- `-A` , `--append-to NAME` , or `--append-set NAME` unions the newly resolved accession set into an existing stored set.
- `--intersection SET` keeps only accessions shared with an explicit accession list and/or `@VARIABLE` reference set before printing or storing.

Use `@` as a prefix when referring to a stored set.

TIP: Stored sets are arguably the most important aspect of selection to grasp. The assemblies and BUSCO results stored in a named set are your dataset. When you are working on a project you should keep track of your dataset through these sets.

5.3 Selector presets

Stored accession sets and selector presets solve related but different problems.

- A stored accession set, such as `@BONOBO` , stores the resolved accessions at the time it was created.
- A selector preset stores the recipe used to resolve accessions, such as “one assembly per genus in Primates”.

This distinction matters. If new assemblies are added later, a stored set stays fixed, while a preset can be rerun against the current database state. In this way they are similar to a view in an SQL database.

Save a selector preset with `selector save` . For example lets say I frequently need to check primate assemblies to see if new assmeblies have been released, or if higher quality assemblies have been released. To do this I may create a preset that captures the best primate assembly per genus and save it as a preset:

```
phyloODB my_project.db selector save primate_refs \  
  --clade Primates \  
  --rank genus \  
  --quantity 1 \  
  --description "One representative assembly per primate genus"
```

Inspect saved presets:

```
phyloODB my_project.db selector list  
phyloODB my_project.db selector show primate_refs  
phyloODB my_project.db selector show primate_refs --json
```

Preview or resolve a preset:

```
phyloODB my_project.db selector preview primate_refs  
phyloODB my_project.db selector resolve primate_refs -S PRIMATE_REFS
```

`preview` prints the accessions that the preset resolves to now. `resolve` does the same resolution and can also freeze the result into a stored accession set.

Delete a preset when it is no longer wanted:

```
phyloODB my_project.db selector delete primate_refs
```

Commands that use the shared selector system accept `--preset NAME` as an input. So now to resolve the preset I can use a list command or a task.

```
phyloODB my_project.db list assemblies --preset primate_refs -p  
phyloODB my_project.db tree --preset primate_refs  
phyloODB my_project.db queue download --preset primate_refs  
phyloODB my_project.db run verify-assembly --preset primate_refs
```

Explicit selector flags supplied alongside `--preset` override or extend the stored recipe. For example:

```
phyloODB my_project.db list assemblies \  
  --preset primate_refs \  
  --downloaded-only \  
  --busco-complete-min 90
```


This keeps the preset as the reusable base recipe while allowing command-specific refinements.

In practice:

- use `@NAME` when you want a fixed accession panel;
- use `--preset NAME` when you want to rerun a reproducible selector recipe;
- use `selector resolve NAME -S PANEL_NAME` when you want to turn a recipe into a fixed panel for downstream work.
- `selector save`, `selector show`, `selector preview`, `selector resolve`, and `selector delete` operate on reusable recipes;
- `assemblies`, `list assemblies`, `tree`, `queue`, and `run` consume those recipes through `--preset NAME`;
- `@NAME` remains the syntax for a stored accession set, not for a stored recipe.

5.4 Exclusions and negative selection

Selectors are not limited to inclusion. If one instead wants to subtract assemblies, use the negative selectors:

- `--exclude-accessions`
- `--exclude-clades`
- `--exclude-taxids`

These are important whenever a rule-based selector is almost correct but a few assemblies are known to be unsuitable.

```
phyloODB my_project.db list assemblies \  
  --clade Primates \  
  --rank genus \  
  --quantity 1 \  
  --exclude-accessions GCF_000001405.40
```

```
phyloODB my_project.db list assemblies \  
  -a @PRIMATES \  
  --exclude-clade "Apes" \  
  --exclude-accession @LEMURS
```

This is often better than abandoning a declarative rule and falling back to a manually typed accession list.

5.5 `list assemblies`: metadata, BUSCO columns, filtering, and output modes

`list assemblies` is the main exploratory command in PhyloODB. By default its output is tab-separated text, so it can be redirected directly into a TSV file.

```
phyloODB my_project.db list assemblies -c Primates -d > primates.tsv
```

A real plain-text example from `production/metazoa.db` is:

```
accession species
GCF_049350105.2 Macaca mulatta
GCF_037993035.2 Macaca fascicularis
GCF_000001405.40 Homo sapiens
```

The same can be done with `>>` when appending to an existing file. Use command `--no-header` to omit the headers. If human readability is preferred in the terminal, `--tidy / -y` aligns columns.

```
phyloODB my_project.db list assemblies -c Primates -d -y
```

Instead of shell redirection, `--output-path` writes the rendered list directly to a file and creates parent directories if needed.

```
phyloODB my_project.db list assemblies \
  --clade Primates \
  --downloaded-only \
  --meta release_date,level \
  --output-path reports/primates.tsv
```

This uses the same output format that would otherwise go to the terminal. With the default output it writes TSV-like text; with `--tidy` it writes the aligned table. If no assemblies match, the file is created or truncated to empty.

Human readable output rendered in the terminal can be produced using `-p` or `--pretty`. By default this will be shown in pages which can be traversed with arrow keys or `[ ]`. It is best treated as a screen mode rather than as a file format. If you want pretty list output by default in a project, set `LIST_USE_COLOR=true`.

```
phyloODB my_project.db list assemblies -c Primates -d -y -p
```

Pretty lists paginate automatically when they exceed the terminal height. Use `[ ]` and `]`, the arrow keys, or Page Up/Page Down to move, Home/End to jump, and `q` to quit. Use `--no-pager` (or `--no-pagination`) to print one complete pretty table and let the terminal's normal scrollbar handle long output.

Showing BUSCO results

`--busco` appends BUSCO summary columns with `--library-name` or `--library-id` so the score source is explicit.

For the parent lineage:

```
phyloODB production/metazoa.db list assemblies \
--clade Primates \
--downloaded-only \
--quantity 5 \
--busco \
--library-name metazoa_odb12 \
--tidy
```

For a custom library:

```
phyloODB production/metazoa.db list assemblies \
--clade Primates \
--downloaded-only \
--quantity 5 \
--busco \
--library-name metazoa_core \
--tidy
```

Example output for `metazoa_core`:

accession	species	complete	single_copy	duplicated
fragmented	hidden_paralog	contaminated	missing	contaminated_assembly
GCF_049350105.2	Macaca mulatta	7.77	0.00	7.77
0.00	NA	92.23	0.00	UNCERTAIN
GCF_037993035.2	Macaca fascicularis	100.00	92.03	7.97
0.00	NA	NA	0.00	NA
GCF_000001405.40	Homo sapiens	98.61	97.61	1.00
1.20	NA	NA	0.20	NA

This is a view that will often be required when building a dataset. Thus there is an alias for `list assemblies --busco --downloaded` with `list results`. A library still needs to be specified.

`--all-runs` with `--busco`

By default, `list assemblies --busco` / `list results` reports **one BUSCO profile** per accession (the current primary run for that accession/library context).

If multiple BUSCO runs exist (for example, different pipelines or formats), add `--all-runs` to expand output to one row per BUSCO run.

In `--all-runs` mode, the score columns are still adjusted through the same screening logic as ordinary BUSCO output. That means:

- the displayed `complete`, `single_copy`, `duplicated`, `fragmented`, `hidden_paralog`, `contaminated`, and `missing` values are resolved for the specific displayed run;
- `--ignore-paralog-filtering` and `--ignore-decontamination` suppress those adjustments in the run-expanded view too;
- `NA` in `hidden_paralog` or `contaminated` means there was no applicable paralog-filtering or decontamination coverage for that accession/run under the requested context, not that the row has fallen back to a different raw display mode.

```
phyloODB my_project.db list results \
  --clade Primates \
  --library-name metazoa_odb12 \
  --all-runs \
  -p
```

To see more information use `list busco-runs`

For practical purposes:

- `miniprot` (- Mi), `metaeuk` (- Me), and `augustus` (- Au) identify the BUSCO pipeline;
- `protein` (P) means the BUSCO run was built from a proteome input, typically `.faa` ;
- `genome` (G) means the BUSCO run was built from a nucleotide assembly input, typically `.fna` .
- `OrthoFinder` (O) means the BUSCO has been reciprocally validated using OrthoFinder. (See Add-Library).

This matters because one accession can legitimately have more than one BUSCO run for the same lineage using the same library. For example the Miniprot pipeline is fast and so may be preferred for a quick pass but once the candidate list is selected it may be preferential to switch to the more comprehensive Augustus pipeline. Furthermore for proteome runs, there may be various proteome profiles with differing isoform cleaning parameters.

5.5.1 Selecting which BUSCO run to use

When more than one BUSCO run exists for an accession, PhyloODB resolves a run context rather than blindly collapsing everything together. The main selector flags are:

- `--busco-pipeline`
- `--require-busco-pipeline`
- `--prefer-busco-pipeline`
- `--format`
- `--require-format`
- `--prefer-format`
- `--busco-run-selection`

The meaning is:

- `--busco-pipeline` and `--require-busco-pipeline` : require a specific pipeline such as `miniprot` , `metaeuk` , or `augustus` ;
- `--prefer-busco-pipeline` : prefer that pipeline while allowing fallback to other matching runs if the pipeline does not exist for an accession within the candidate

set;

- `--format` and `--require-format` : require a specific BUSCO input format, `protein` or `genome` ;
- `--prefer-format` : prefer one BUSCO input format while allowing fallback;
- `--busco-run-selection` : choose whether the selected run should come from the current `primary` run pointer or the `latest` run in that context.

Note: the hard BUSCO filters do not override `--busco-run-selection` . If you ask for `--busco-pipeline augustus --busco-run-selection primary` , PhyloODB will only use Augustus runs that are already stored as the current primary for each accession. If you instead mean “use the latest Augustus run if one exists”, use `--busco-run-selection latest` .

Require and prefer are different:

- require filters shrink the candidate run set;
- prefer filters only affect ranking inside the already valid candidate set;
- if no preferred match exists, PhyloODB falls back to the best non-preferred candidate that still satisfies the required filters.

Typical examples:

Show BUSCO columns using MetaEuk runs with genome input. If the matching primary run is also MetaEuk/genome, it is preferred; otherwise PhyloODB falls back to another matching MetaEuk genome run, usually the latest. Primary runs from other pipelines, or with protein input, are ignored for this display.

```
phyloODB my_project.db list assemblies \  
  --accessions @TARDIGRADES \  
  --busco \  
  --library-name ecdysozoa_core \  
  --busco-pipeline metaeuk \  
  --format genome \  
  --busco-run-selection primary \  
  --tidy
```

Prefer MetaEuk genome runs, but allow fallback to other matching runs if a given accession does not have one:

```
phyloODB my_project.db list assemblies \  
  --accessions @TARDIGRADES \  
  --busco \  
  --library-name ecdysozoa_core \  
  --prefer-busco-pipeline metaeuk \  
  --prefer-format genome \  
  --tidy
```

Inspect all candidate runs while still constraining to one pipeline and one input format:

```
phyloODB my_project.db list assemblies \  
  --accessions @TARDIGRADES \  
  --busco \  
  --library-name ecdysozoa_core \  
  --all-runs \  
  --busco-pipeline metaeuk \  
  --format genome \  
  --tidy
```

This is the right way to ask a concrete question such as “show me the MetaEuk genome BUSCO results for these accessions” rather than “show me whichever BUSCO row happens to be primary today”.

The practical rule is:

- use `primary` when you want the currently curated default run;
- use `latest` when you want the most recently registered run in a matching context;
- use `--all-runs` when you are auditing or debugging the run landscape itself.

BUSCO-aware selectors can also filter on whether BUSCO data exist at all:

- `--has-busco-results` : keep only accessions that have BUSCO results in the requested library context.
- `--missing-busco-results` : keep only accessions that do not yet have BUSCO results in that context.

These are useful for queueing only missing work or auditing coverage:

```

phyloODB my_project.db list assemblies \
  --accessions @TARGETS \
  --library-name metazoa_odb12 \
  --missing-busco-results \
  --store TARGETS_MISSING_METAZOA_BUSCO

phyloODB my_project.db queue batch-busco \
  --accessions @TARGETS_MISSING_METAZOA_BUSCO \
  --lineage metazoa_odb12

```

Selectors can also filter by previous screening state:

- `--paralog-filtered` and `--not-paralog-filtered`
- `--min-hidden-paralogs` and `--max-hidden-paralogs`
- `--decontaminated` and `--not-decontaminated`
- `--contaminated`
- `--decontamination-run`
- `--ignore-contaminated-assemblies` and `--include-contaminated-assemblies`

Use these when you want to inspect or queue work against a panel based on what has already passed through paralog filtering or decontamination. For example:

```

phyloODB my_project.db list results \
  --accessions @TARGETS \
  --library-name metazoa_core \
  --paralog-filtered \
  --decontaminated \
  --tidy

```

5.5.2 Metadata

All assemblies downloaded from NCBI will have metadata attached. This includes comments, release-date, publisher, etc. All the available fields to view can be found by issuing the `list metadata` command.

`--meta` or `-m` appends assembly metadata columns. With no value, the defaults are `release_date`, `level`, `n50`, and `comments`. `origin` is also available as an explicit metadata field and records values such as `local`, `refseq`, or `genbank`.


```

phyloODB production/metazoa.db list assemblies \
  -c Primates \
  -d \
  -q 5 \
  -m \
  -b \
  -l metazoa_odb12 \
  -y

```

Example output from `production/metazoa.db` :

accession	species	release_date	level	n50
comments				
complete	single_copy	duplicated	fragmented	contaminated
missing				
GCF_049350105.2	Macaca mulatta	2025-07-24	Complete Genome	
162285572				
6.70	0.00	6.70	0.00	93.30
0.00				
GCF_037993035.2	Macaca fascicularis	2025-03-18	Complete Genome	
162126771				
100.00	92.86	7.14	0.00	NA
0.00				
GCF_000001405.40	Homo sapiens	2022-02-03	Chromosome	
57879411	Genome Reference Consortium Human Build 38 patch release 14			
(GRCh38.p14)	98.66	97.92	0.74	1.19
0.15				NA

An explicit metadata list can also be supplied.

```

phyloODB my_project.db list assemblies \
  --clade Primates \
  --downloaded-only \
  --meta release_date,level,n50,comments

```

5.5.3 Filtering Assemblies

`--filter` is a general expression mechanism spanning metadata and BUSCO results.

- `,` means AND
- `|` means OR
- repeat `--filter` to add more AND clauses
- BUSCO fields use the `busco.` prefix

Examples:

```

phyloODB my_project.db list assemblies \
  --clade Primates \
  --filter "level contains chrom"

phyloODB my_project.db list assemblies \
  --clade Primates \
  --library-name metazoa_odb12 \
  --filter "busco.complete>=90,busco.single_copy_complete>=80" \
  --busco \
  --tidy

```

5.5.4 Sorting Assemblies

Use `-s / --sort` to order output rows. Append `:desc` or `:asc` for descending or ascending order. When listing assemblies, useful aliases include `latest` for `release_date:desc` and `quality` for BUSCO single-copy completeness in descending order.

```

phyloODB my_project.db list assemblies \
  --clade Primates \
  --meta release_date,level \
  --sort latest

phyloODB my_project.db list results \
  --accessions @CORE_SET \
  --library-name metazoa_core \
  -s busco.complete:desc,busco.single_copy:desc

```

One caveat worth knowing: sorting only works on columns available in that output. So `latest` needs `release_date` in the output, and `quality / busco.*` sorting needs BUSCO columns, as in `list results` or `list assemblies --busco`.

5.6 Listing BUSCO runs and Gene Families

`list assemblies --busco` is useful for assembly-centric review.

For run-centric inspection and per-family inspection, use:

- `list busco-runs` : one row per BUSCO run record.
- `list buscos` : BUSCO family-level rows from selected runs.
- `list proteome-profiles` : one row per registered proteome profile and its preparation provenance.

In general this is more for specific refinements and is not necessary for ordinary use.

Examples:

```
phyloODB my_project.db list busco-runs \  
  --library-name metazoa_odb12 \  
  --clade Primates \  
  --prefer-busco-pipeline metaeuk \  
  --prefer-format genome \  
  --tidy -c  
  
phyloODB my_project.db list busco-runs \  
  --accessions GCF_000001405.40 \  
  --run-id 1234 \  
  --tidy  
  
phyloODB my_project.db list busco-runs \  
  --accessions @TARDIGRADES \  
  --busco-pipeline augustus \  
  --ids-only  
  
phyloODB my_project.db list busco-runs \  
  --accessions @TARDIGRADES \  
  --busco-pipeline augustus \  
  --ids-only \  
  --store-results AUGUSTUS_RUNS  
  
phyloODB my_project.db list buscos \  
  --library-name metazoa_odb12 \  
  --accessions GCF_000001405.40 \  
  --run-id 1234 \  
  --tidy  
  
phyloODB my_project.db list buscos \  
  --library-name metazoa_odb12 \  
  --family-id BUSCO_0010450 \  
  --tidy  
  
phyloODB my_project.db list proteome-profiles \  
  --accessions GCA_000516915.1 \  
  --tidy
```

Practical notes:

- `--run-id` pins output to a specific BUSCO run.
- `--family-id` filters `list buscos` to one or more BUSCO families.
- `--ids-only` emits the filtered BUSCO run ids as a comma-separated list.

- `--store-results NAME` stores the filtered BUSCO run ids in a database variable for later reuse.
- `--tidy`, `-c/--colour`, and TSV default output modes work for these list routes in the same way as `list assemblies`.
- Stored run-id variables are distinct from stored accession sets. They can be reused on run-id aware commands such as `set busco-primary --run-ids @AUGUSTUS_RUNS`.

5.7 Proteome profiles in listing and selection

When a proteome is downloaded or imported, PhyloODB can automatically queue proteome preparation. This creates a derived proteome profile rather than mutating the raw proteome. The default recipe is controlled by the database `DEFAULT_PROTEOME_*` variables. In a new database, `DEFAULT_PROTEOME_USE_GFF=true` and `DEFAULT_PROTEOME_USE_CDHIT=false`, so automatic preparation uses gene ids from the associated `.gff` file when available and skips CD-HIT unless the project default or command flags enable it.

The result is that a given accession can have several BUSCO runs from different proteome variants. Naturally only one proteome profile is the accession default for later proteome-aware workflows. Automatic preparation normally sets the prepared profile as default (`DEFAULT_PROTEOME_SET_DEFAULT=true`), and you can curate it later with `set proteome-profile`.

Examples of filtering by proteome-profile status:

```
phyloODB my_project.db list busco-runs \
  --accessions GCA_000516915.1 \
  --tidy

phyloODB my_project.db list assemblies \
  --busco --all-runs \
  --accessions GCA_000516915.1 \
  --meta proteome_profile,default_proteome_profile \
  --tidy
```

Selector flags for proteome-aware commands:

- `--proteome-profile NAME` : require a specific proteome profile.

- `--prefer-proteome-profile NAME` : prefer that profile while allowing fallback.
- `--isoforms-cleaned` : shortcut for "use the accession's current default cleaned proteome profile".
- `--raw-proteome` : shortcut for `--proteome-profile raw` .

Proteome profile display is abbreviated for readability:

- `raw`
- `gff`
- `cdhit96`
- `gff,cdhit96`

An asterisk suffix indicates the accession's current default proteome profile, for example `gff,cdhit96*` .

Important distinction:

- The stored profile name is the real profile identifier, for example `raw` , `cdhit96` , `gff` , or `gff_cdhit96` .
- In user-facing tables, recipe-style display labels such as `gff,cdhit96` are derived from preparation provenance.

6. Variables and System Defaults

6.1 Variables and manual configuration

PhyloODB stores project variables in the `Environment_Variables` table. These are exposed through:

```
phyloODB my_project.db list variables
phyloODB my_project.db list variables --json > variables.json
phyloODB my_project.db set var VARIABLE value
phyloODB my_project.db set var --json variables.json
phyloODB my_project.db set env VARIABLE value
phyloODB my_project.db set proteome-profile ...
phyloODB my_project.db set busco-primary ...
```

`set var` uppercases the key and attempts to parse the value as JSON. This means the following are all valid:

```
phyloODB my_project.db set var SELECTOR_DEFAULT_DOWNLOADED_ONLY true
phyloODB my_project.db set var DAEMON_PROCESS_POLLING_TIME 1.5
phyloODB my_project.db set var MY_PANEL
'["GCF_000001405.40","\GCF_037993035.2"]'
phyloODB my_project.db set var NOTE "\"manual review pending\""
```

Variables are typed in the database as one of:

- `env` : configuration, paths, defaults, notes, and task pointers.
- `assemblies` : stored accession/assembly sets. These are displayed with an `@` prefix by `list variables`.
- `busco_runs` : stored BUSCO run-id sets.

When an older database is migrated, PhyloODB backfills the type from the existing value. New `set var` writes also infer a type by default: all-numeric lists become BUSCO run-id sets, accession-like lists become assembly sets, and everything else becomes `env`. Known system configuration names are kept as `env` even if their value is a numeric list.

You can override inference explicitly:

```

phyloODB my_project.db set var --kind assemblies REVIEW_PANEL
'["GCF_000001405.40"]'
phyloODB my_project.db set var --kind busco-runs AUGUSTUS_RUNS
'[101,202]'
phyloODB my_project.db set var --kind env DEFAULT_THREADS_BUSCO_RUN 4
phyloODB my_project.db set var --kind env SET_MAX_THREADS_ON_START false

```

Variables can also be exported and imported as strict JSON:

```

phyloODB my_project.db list variables --json > variables.json
phyloODB my_project.db list variables --kind assemblies --json >
panels.json
phyloODB my_project.db set var --json variables.json

```

The JSON format is split into explicit objects:

- `environment` : configuration, paths, defaults, notes, and task pointers.
- `assemblies` : stored accession sets, equivalent to the `@NAME` panels used by selectors.
- `busco_runs` : stored BUSCO run-id sets.

Importing JSON writes each object to its matching database kind. The importer ignores `_metadata` and `_definitions`, so exported files can include definitions that explain the variables without changing the stored values. See `docs/variables.example.json` for a complete example with definitions for the environment variables used by PhyloODB.

Legacy `set VARIABLE=value` and `set VARIABLE value` still work, but they are deprecated. Use `set var` or `set env` instead.

In practice the variables fall into several categories.

- Path and tool configuration: `GENOME_DIR`, `LIBRARIES_DIR`, `BUSCO_BINARIES_PATH`, `ORTHOFINDER_OUTPUT_DIR`, `BLASTP_PATH`, `MAKEBLASTDB_PATH`.
- Logging and list display: `LOG_DIR`, `LOG_LEVEL`, `LOG_TO_CONSOLE`, `LOG_MAX_BYTES`, `LOG_BACKUPS`, `LIST_USE_COLOR`.
- Daemon configuration: `DAEMON_MAX_THREADS`, `SET_MAX_THREADS_ON_START`, `DAEMON_PROCESS_POLLING_TIME`, `BLOCKED_TASK_QUEUE_POLLING_TIME`, and task-specific defaults such as `DEFAULT_THREADS_BUSCO_RUN`.

- BUSCO pipeline defaults and tuning:
 - general: `DEFAULT_BUSCO_FORMAT` , `DEFAULT_BUSCO_PIPELINE` , `BUSCO_MINIPROT_KEEP_REF_FILE`
 - Augustus: `BUSCO_AUGUSTUS_EVALUATE` , `BUSCO_AUGUSTUS_LIMIT` , `BUSCO_AUGUSTUS_LONG` , `BUSCO_AUGUSTUS_SPECIES` , `BUSCO_AUGUSTUS_PARAMETERS`
 - Metaeuk: `BUSCO_METAUEK_PARAMETERS` , `BUSCO_METAUEK_RERUN_PARAMETERS`
 - Miniprot: `BUSCO_MINIPROT_PARAMETERS`
- Proteome preparation defaults: `DEFAULT_PROTEOME_CLEAN_ISOFORMS` , `DEFAULT_PROTEOME_USE_GFF` , `DEFAULT_PROTEOME_USE_CDHIT` , `DEFAULT_PROTEOME_CDHIT_IDENTITY` , `DEFAULT_PROTEOME_MAX_CONCURRENT` , `DEFAULT_PROTEOME_THREADS_PER_JOB` .
- Selector defaults: `SELECTOR_DEFAULT_DOWNLOADED_ONLY` , `SELECTOR_DEFAULT_PRIMARY_ONLY` , `SELECTOR_DEFAULT_USE_BUSCO` , `SELECTOR_DEFAULT_STATUS_MIN` , `SELECTOR_DEFAULT_PROTEIN_ONLY` , `SELECTOR_SCORE_ORDER` , `SELECTOR_BUSCO_BUCKETS` .
- Stored accession sets: user-defined panels such as `METAZOA_CORE` or `MAMMALS` .
- Stored BUSCO run-id sets: user-defined run lists such as `AUGUSTUS_RUNS` .
- Convenience pointers maintained automatically by the CLI and tasks: `LAST` , `LAST_DOWNLOAD_ASSEMBLIES` , `LAST_PARALOG_REMOVAL` , `ACTIVE_DECONT_RUN_<library_id>` , `ACTIVE_INTERNAL_DECONT_RUN_<library_id>` .

Thread defaults are stored as ordinary environment variables. `DAEMON_MAX_THREADS` is the total daemon/run thread budget, while `DEFAULT_THREADS_<TASK>` variables set the default thread request for individual registered tasks. For example, `DEFAULT_THREADS_BUSCO_RUN` controls the default `busco-run` task thread count and `DEFAULT_THREADS_ORTHOFINDER_RUN` controls `orthofinder-run` .

By default `SET_MAX_THREADS_ON_START=true` , so `run` and daemon startup recalculate `DAEMON_MAX_THREADS` from the currently detected allocation and refresh task defaults. This is useful on clusters where a database may be reused inside jobs with different CPU allocations. Set it to `false` if you want stored thread settings to remain fixed:


```
phyloODB my_project.db set var SET_MAX_THREADS_ON_START false
phyloODB my_project.db set var DEFAULT_THREADS_BUSCO_RUN 4
```

An excerpt from `production/metazoa.db list variables` illustrates the mixture:

```
GENOME_DIR
"/home/q122514/PhyloODB/production/genomes"
LIBRARIES_DIR
"/home/q122514/PhyloODB/production/libraries"
DAEMON_MAX_THREADS          112
DEFAULT_THREADS_BUSCO_RUN    8
DEFAULT_THREADS_ORTHOFINDER_RUN 24
DAEMON_PROCESS_POLLING_TIME  2
LAST                         1738
LAST_PARALOG_REMOVAL        1738
ACTIVE_DECONT_RUN_1          {"run_id":
"idc_6fbb77e4c4057932bd0bc15ed4ad19600d093ab7", ...}
METAZOA_CORE                  ["GCA_000328365.1", "GCF_000001405.40",
..., "SDOL_2024"]
AUGUSTUS_RUNS                  ["535", "1186", "2448"]
```

The practical lesson is that PhyloODB is highly configurable. Many behaviours are driven by values visible in `list variables`, not by hidden process state.

6.2 Default proteome profiles

Proteome profiles are separate from BUSCO primary runs.

- `set proteome-profile` changes the accession's default proteome profile.
- `set busco-primary` changes which BUSCO run is primary for an accession/library context. Runs on the same assembly may differ by proteome profile, as well as pipeline or other aspects.

Examples:

```
phyloODB my_project.db list proteome-profiles \  
  --accessions GCA_000516915.1 \  
  --tidy  
  
phyloODB my_project.db set proteome-profile \  
  --accessions GCA_000516915.1 \  
  --profile-name cdhit96 \  
  --dry  
  
phyloODB my_project.db set proteome-profile \  
  --accessions GCA_000516915.1 \  
  --profile-name cdhit96
```

The default proteome profile is what proteome-aware commands use when you do not pass `--proteome-profile` explicitly. This makes it possible to keep several derived proteomes, run BUSCO on all of them, then decide both:

- which BUSCO run should be primary;
- which proteome profile should become the accession default.

6.3 Manual BUSCO primary overrides

Primary BUSCO selection is automatic by default. PhyloODB maintains a stored `primary` run per accession, library, and purpose (`default` , `export_protein` , `export_nucleotide`) and refreshes those automatic primaries to the best usable run whenever BUSCO discovery, BUSCO task completion, or BUSCO verification updates the run set. Certain genome pipelines will not produce nucleotide output (miniprot).

Automatic ranking prefers:

1. higher single-copy complete count;
2. then higher total complete count;
3. then lower duplicated count;
4. then later completion time;
5. then higher run id.

When you need to pin a run manually, use `set busco-primary` .

Examples:

```
phyloODB my_project.db set busco-primary \  
  --accessions @TARDIGRADES \  
  --busco-pipeline augustus \  
  --format genome \  
  --dry
```

```
phyloODB my_project.db set busco-primary \  
  --accessions @TARDIGRADES \  
  --format protein
```

```
phyloODB my_project.db set busco-primary \  
  --accession GCA_000001405.1 \  
  --run-id 535
```

```
phyloODB my_project.db set busco-primary \  
  --accessions @TARDIGRADES \  
  --run-ids @AUGUSTUS_RUNS
```

If you want to recompute automatic primaries rather than pin a manual override, use `set busco-primary --refresh`.

Examples:

```
phyloODB my_project.db set busco-primary --refresh  
phyloODB my_project.db set busco-primary --refresh --accessions  
@TARDIGRADES --library-name arthropoda_odb12  
phyloODB my_project.db set busco-primary --refresh --dry --accessions  
ACC1,ACC2
```

Refresh mode:

- operates on accession/library pairs that already have completed `BUSCO_Runs` ;
- preserves existing manual overrides (`policy=manual_override`);
- recomputes `default` , `export_protein` , and `export_nucleotide` ;
- treats omitted selectors as database-wide scope;
- cannot be combined with manual pinning flags such as `--run-id` , `--run-ids` , `-format` , or `--busco-pipeline` .

Important rules:

- if you do not supply `--refresh` , then without `--run-id` or `--run-ids` you must provide at least one run-disambiguating selector such as `--format` or `--busco-`

`pipeline ;`

- the selected run updates every primary purpose that the run can actually support;
- this is based on the chosen run's capabilities, not on the value of `--format` alone;
- for example, a protein-input `miniprot` run will usually update `default` and `export_protein`, while a genome-input `metaeuk` or `augustus` run will usually update all three purposes.

`--dry` prints the current primary assignments and the proposed replacements without writing anything.

Manual overrides are persistent. Once a slot has been set manually, automatic BUSCO completion, discovery, and verify repair will not overwrite it.

To remove manual overrides and return those accessions to automatic best-run selection:

```
phyloODB my_project.db purge busco-primary --accessions @TARDIGRADES
phyloODB my_project.db purge busco-primary --accessions @TARDIGRADES --
apply
```

The purge is dry-run by default. With `--apply`, PhyloODB removes only manual BUSCO primary overrides and immediately recomputes automatic best primaries for the affected accession/library pairs.

7. Queueing, running, and the daemon model

Every analysis task in PhyloODB can generally be launched in one of two ways.

```
phyloODB my_project.db queue <task> ...  
phyloODB my_project.db run <task> ...
```

`run` is immediate and foregrounded. It is suitable for direct experimentation or for simple jobs.

`queue` records the task in the database for execution by the daemon. This mode is central to the intended operating model of PhyloODB because many tasks depend on earlier tasks and may need to wait for data or resources.

7.1 Runtime thread defaults

PhyloODB separates the total thread budget from each task's default thread request.

- `DAEMON_MAX_THREADS` is the maximum total worker-thread budget for a daemon or foreground `run`.
- `DEFAULT_THREADS_<TASK>` variables define the default thread request for individual registered tasks.
- Explicit task `--threads N` overrides the task default for that command.
- Explicit daemon `--threads N`, `--max-threads N`, or `--max-concurrent N` overrides the stored `DAEMON_MAX_THREADS` for that daemon invocation.

When `SET_MAX_THREADS_ON_START=true` (the default), PhyloODB recalculates the detected available thread count whenever a daemon starts or a foreground `run` starts. It then refreshes `DAEMON_MAX_THREADS` and the generated `DEFAULT_THREADS_<TASK>` values. This is intended for clusters and schedulers where the same database may be used inside jobs with different CPU allocations. It is recommended that if you are running a submission script on a cluster you use `run` commands sequentially or in parallel rather than relying on the daemon and queue system.

The detected count is treated as a hard upper bound. If a stored `DAEMON_MAX_THREADS` value, daemon `--threads`, or daemon `--max-threads` asks for more threads than PhyloODB detects as available, startup fails with an error.

The thread variable is autogenerated from the task name.

- `busco-run` -> `DEFAULT_THREADS_BUSCO_RUN`
- `orthofinder-run` -> `DEFAULT_THREADS_ORTHOFINDER_RUN`
- `BatchBuscoTask` -> `DEFAULT_THREADS_BATCH_BUSCO_TASK`

A few high-cost tasks have task-owned rules:

- `busco-run` : adaptive defaults of 1, 2, 4, or 8 threads depending on the daemon budget, so normal allocations can run at least two BUSCO jobs concurrently.
- `orthofinder-run` : `min(DAEMON_MAX_THREADS, 24)` .

If a `DEFAULT_THREADS_<TASK>` variable is missing, `null` , invalid, or non-positive, PhyloODB falls back to the task registry default and logs a warning. The final value is still capped to the daemon's thread budget. Setting `SET_MAX_THREADS_ON_START` to `false` will allow you to fully customise the threads allocated to different tasks.

Examples:

```
phyloODB my_project.db set var DEFAULT_THREADS_BUSCO_RUN 4
phyloODB my_project.db set var DEFAULT_THREADS_ORTHOFINDER_RUN 16
phyloODB my_project.db set var SET_MAX_THREADS_ON_START false

phyloODB my_project.db run busco --threads 4 ...
phyloODB-daemon my_project.db start --here --threads 32
```

7.2 Scheduling and dependencies

Queued tasks can be blocked until a condition is satisfied by using `--schedule` .

Common expressions are:

- `started:LAST`
- `finished:LAST`
- `succeeded:LAST`
- `failed:<task_id>`
- `delay:30s`
- `at:02:00`
- `queued-drained`

This allows command chains such as:

```
phyloODB my_project.db queue download-busco-library --lineage
metazoa_odb12 --coverage 1
phyloODB my_project.db queue --schedule succeeded:LAST download-busco-
library --lineage mammalia_odb12 --coverage 1
```

For shell scripts and workflow managers, capture the task id and then wait for a terminal status:

```
task_id=$(phyloODB my_project.db queue download --print-id --accessions
@PANEL)
phyloODB my_project.db status "$task_id" --wait --quiet
case "$?" in
  0) echo "download completed" ;;
  1) echo "download failed" >&2; exit 1 ;;
  2) echo "download still pending" >&2; exit 2 ;;
  3) echo "unknown task id" >&2; exit 3 ;;
  4) echo "could not check task status" >&2; exit 4 ;;
esac
```

`status` also accepts selectors such as `LAST` and `LAST_DOWNLOAD`, matching the selectors used by schedule expressions. Its exit-code is: `0` completed, `1` failed, `2` incomplete or timed out, `3` unknown task selector/id, and `4` status-check error.

7.3 The daemon and queue inspection

The daemon is a separate entry point:

```
phyloODB-daemon <database> {start,stop}
```

Its job is to monitor the `Tasks` table, respect dependencies and parent-child relationships, and launch runnable tasks as resources allow. In other words, `queue` writes intentions into the database, and `phyloODB-daemon` turns those intentions into executed jobs.

Typical control commands are:

```
phyloODB-daemon my_project.db start
phyloODB-daemon my_project.db start --log-console --log-level INFO
phyloODB-daemon my_project.db stop
phyloODB-daemon my_project.db stop --drain
```

Important runtime controls include:

- `--max-threads` , `--max-concurrent` , or `--threads`
- `--polling`
- `--blocked-polling`
- logging controls such as `--logfile` , `--log-level` , `--color`

Thread limits follow the runtime thread default rules above: explicit daemon thread limits override `DAEMON_MAX_THREADS` , but they must not exceed the detected available thread count.

When the daemon is running, add `--watch` to the queue or error list:

```
phyloODB my_project.db list queue -w
phyloODB my_project.db list queue --watch --all --refresh 1
phyloODB my_project.db list errors --watch
phyloODB my_project.db list errors --watch --stack --limit 50
```

`watch queue` and `watch errors` are exact aliases for these commands.

```
phyloODB my_project.db watch queue
phyloODB my_project.db watch errors
```

Task queue ordering is controlled with `list queue -s ...` or `list queue --sort ...` . The default is

`--sort latest` , which sorts each top-level task block by the most recent status change anywhere in its parent/subtask tree. If a subtask changes, the whole parent block moves up, and children inside that block are sorted recursively so the changed child is easy to find. Other profiles are `new` , `old` , `errors` , `running` , and `status` ; aliases include `changed` , `newest` , `oldest` , and `active` .

Library and storage file moves are stored in a recovery journal.

If a move is interrupted after its database metadata commits, inspect it with:


```
phyloODB my_project.db storage recover
```

Retry safe finalization only after reviewing the paths:

```
phyloODB my_project.db storage recover --operation-id 12 --apply
```

Example snapshot from `production/metazoa.db`:

Tasks Queue								
Task	Task					Queue	Start	
ID	Name	Prio...	C	Status	Why	Time	Time	
1666	TaskD...	1	0	R		14:4...	14:49...	
1667	add-l...	3	3	S		15:0...	15:00...	
1668	ortho...	1	0	R		15:0...	15:00...	
(1667)								
1738	paral...	3	1	R		16:0...	16:03...	

This queue/error display is often the most efficient way to understand what the project is doing at a given moment.

8. PhyloODB Tasks

PhyloODB exposes a broad task catalogue. In practice these tasks belong to a smaller number of conceptual phases.

For all tasks information on the flags and arguments can be found by following the task with a `--help / -h`.

8.1 Data acquisition and registration

Relevant tasks:

- `create-taxonomy`
- `update-assembly`
- `download-assemblies`
- `import-local-assembly`
- `batch-import-local-assembly`
- `verify`
- `verify-downloads`
- `verify-busco`
- `verify-libraries`
- `verify-orthofinder`
- `split-records`
- `prepare-proteome`

These tasks establish what data exist and where sequence files live.

8.1.1 Taxonomy and assembly metadata

`create-taxonomy` populates the database taxonomy tables from an NCBI taxdump. It is normally run during `create` or `reset`, but can be rerun periodically to repair or update taxonomy knowledge. It can be pointed to an existing taxdump.

`update-assembly` fetches assembly metadata from NCBI for a taxid, clade-derived taxid, or explicit accession list. It records what assemblies exist and their metadata, but it does not download genome or proteome sequence files. This is the task behind the common `queue add --clade ...` pattern introduced earlier in the manual.

```
phyloODB my_project.db queue add --clade Primates
phyloODB my_project.db run add --accessions
GCF_000001405.40,GCF_009914755.1
```

After this step, `list assemblies` can inspect and filter available assemblies, but the sequence files are not yet local unless they were already imported or discovered.

8.1.2 Downloading assemblies

`download-assemblies` is the standard route for bringing selected assemblies into the local project. Downloads from NCBI are staged first, checked against the assembly FTP `md5checksums.txt` manifest where available, and fully read as gzip streams before they are accepted. The checksum manifest is stored as a genome artifact, and expected file checksums are recorded on the downloaded FNA/FAA/GFF artifacts.

Downloads can be launched from stored accession sets, explicit accessions, or selectors:

```
phyloODB my_project.db run download --accessions @PRIMATE_REFS

phyloODB my_project.db queue download \
  --clade Primates \
  --rank genus \
  --quantity 1
```

Downloaded files are written under the active `genomes` storage root. If multiple genome roots are configured, only the active root is used for new downloads; inactive roots remain valid for existing assemblies already bound there.

8.1.3 Automatic proteome preparation after download

Download, local import, batch local import, and some verification workflows can automatically queue proteome preparation. This creates a derived proteome profile rather than mutating the raw downloaded/imported proteome.

Automatic preparation follows the database `DEFAULT_PROTEOME_*` variables. In a new database:

- `DEFAULT_PROTEOME_CLEAN_ISOFORMS=true`

- `DEFAULT_PROTEOME_USE_GFF=true`
- `DEFAULT_PROTEOME_USE_CDHIT=false`
- `DEFAULT_PROTEOME_SET_DEFAULT=true`

This means automatic preparation uses GFF-based isoform reduction where available, skips CD-HIT by default, and makes the prepared profile the accession's default proteome profile. To make CD-HIT part of the project default, set:

```
phyloODB my_project.db set var DEFAULT_PROTEOME_USE_CDHIT true
phyloODB my_project.db set var DEFAULT_PROTEOME_CDHIT_IDENTITY 0.98
```

To override the default for one download/import task, use the `clean-*` options, for example:

```
phyloODB my_project.db queue download \
  --accessions @PRIMATE_REFS \
  --no-clean-skip-cdhit \
  --clean-cdhit-identity 0.98
```

When automatic `prepare-proteome` work is queued, it is treated as a required subtask. The parent download/import/verify task waits for it through the normal suspension/resume system, and a preparation failure is reflected on both the parent and child task.

`prepare-proteome` can also be run explicitly. At a high level it uses annotation and/or CD-HIT-based logic to create an immutable derived proteome profile from the raw imported or downloaded proteome. This can then be set as default through the `set proteome profile` with names obtained from `list proteome-profiles`.

```
phyloODB my_project.db queue prepare-proteome \
  --accessions @PRIMATE_REFS \
  --profile-name gff_cdhit98 \
  --no-skip-cdhit \
  --cdhit-identity 0.98
```

8.1.4 Local import integrity

Local imports use the same integrity model as downloads. Because there is no upstream NCBI manifest for a local file, PhyloODB writes `phyloodb_md5checksums.txt` beside the imported assembly files after validating gzip input and computing MD5 values. This local manifest has the same format and function as the NCBI manifest: it is registered as a genome artifact, and its checksums are stored on the corresponding FNA/FAA/GFF artifact rows. It proves that files still match what PhyloODB originally imported.

`split-records` is a special recovery/import helper for genome folders that contain multiple independent FASTA records or isolated proteomes that should become separate accessions. Most users encounter it indirectly through verification options such as `--split-isolated-proteomes`; use it deliberately when a single imported folder needs to be decomposed into multiple accession records.

8.1.5 Local import and automatic isoform cleaning

Both `import-local-assembly` and `batch-import-local-assembly` follow the same `DEFAULT_PROTEOME_*` variables as download tasks do. In a new database, local import should therefore be understood as “import, then clean using GFF reduction where available, without CD-HIT unless explicitly enabled”.

Single local import requires at least one of `--fna` or `--faa`, and taxonomic identification via `--taxid`, `--taxon-name`, or `--genus` with `--species`.

Examples:

```
phyloODB my_project.db queue import-local-assembly \
  --faa /data/local/GCF_999999999.1/proteins.faa.gz \
  --gff /data/local/GCF_999999999.1/annotations.gff3.gz \
  --accession GCF_999999999.1 \
  --taxid 9606

phyloODB my_project.db queue import-local-assembly \
  --fna /data/local/sample/genome.fna.gz \
  --faa /data/local/sample/proteins.faa.gz \
  --genus Pan \
  --species troglodytes \
  --metadata '{"assembly_level":"chromosome"}'
```

Local imports write `Assembly.origin=local` automatically. Assemblies discovered from NCBI metadata write `origin=refseq` or `origin=genbank` based on the accession.

Useful local-import controls include:

- `--copy-to-genome-dir` or `--no-copy-to-genome-dir`
- `--location`
- `--clean-isoforms` or `--no-clean-isoforms`
- `--skip-clean-isoforms`
- `--clean-skip-gff`
- `--clean-skip-cdhit` or `--no-clean-skip-cdhit`
- `--clean-gff-priority`

Batch local import expects a directory and can optionally restrict which accessions in that directory should be imported.

```
phyloODB my_project.db queue batch-import-local-assembly \  
  --assembly-dir /data/local_batch \  
  --accessions-for-import GCF_999999999.1 GCF_999999998.1
```

If a user wants to preserve raw imported proteomes temporarily, they should disable or adjust cleaning explicitly rather than assuming import is passive.

8.1.6 The verify tasks

The verify tasks are maintenance and recovery tools rather than workflow steps. They are especially useful when moving roots; see [storage roots and recovery](#).

`verify` is the orchestration task. It runs the relevant verification suite across assemblies, libraries, BUSCO runs, and OrthoFinder runs. Use it when you want a broad project audit rather than a targeted check.

`verify-downloads` is used to reconcile the database with files on disk. Depending on flags, it can:

- check for missing or corrupt downloads, including stored NCBI or PhyloODB-local checksums where present;
- discover files that exist on disk but are not reflected properly in the database;
- reacquire missing files with `--reacquire` ;
- tidy or organise folder layout with `--tidy` , `--organise` , and `--organise-check-only` ;

- detect isolated proteomes and optionally split them with `--split-isolated-proteomes` ;
- run or revert isoform-cleaning-related checks with the `clean-*` options.

Example:

```
phyloODB my_project.db queue verify-downloads \
  --clade Primates \
  --downloaded-only \
  --discover \
  --report verify_downloads.tsv
```

`verify-busco` does the same kind of audit for BUSCO results. Its main roles are:

- `--discover` : find results already present on disk;
- `--reingest` : parse and re-register BUSCO outputs into the database;
- `--queue-missing` : submit BUSCO jobs for accessions that should have results but do not;
- `--report` : write a verification report.

Verify-task reports are written under the shared reports root by default, in the `verify-reports/task_<task_id>_<timestamp>.../` namespace. `--report` still overrides the specific report filename when you want an explicit path.

`verify-busco --reingest` understands run-centric BUSCO layouts and records run metadata in the BUSCO run tables.

Going forward, run folders should follow the pipeline-aware naming pattern

`run_<pipeline>_<lineage>` (for example `run_miniprot_metazoa_odb12` or `run_augustus_metazoa_odb12`).

Example:

```
phyloODB my_project.db queue verify-busco \
  --clade Primates \
  --library-name metazoa_odb12 \
  --discover \
  --queue-missing \
  --report verify_busco.tsv
```

`verify-libraries` audits BUSCO lineage and derived library records. In repair mode it can backfill library artifacts, refresh stale state, and reconcile library folders after a storage move.

`verify-orthofinder` audits OrthoFinder runs, backfills artifacts, and can mark broken runs unusable. `verify-orthofinder --repair` expects an active `orthofinder` storage root so that OrthoFinder result directories can be registered canonically as rooted artifacts.

8.2 BUSCO and orthology preparation

Relevant tasks:

- `download-busco-library`
- `busco-run`
- `batch-busco`
- `orthofinder-run`
- `add-library`
- `import-custom-library`
- `mafft-run`
- `iqtree-run`
- `annotate-orthogroup-tree`

This group covers running BUSCO on assemblies, building project-specific orthology libraries, and inspecting the internal alignments and trees used during library construction.

8.2.1 BUSCO lineage libraries

`download-busco-library` registers a BUSCO lineage dataset for use in later analysis. This is often the earliest analysis task in a project.

```
phyloDB my_project.db queue download-busco-library \  
  --lineage metazoa_odb12  
  
phyloDB my_project.db run download-busco-library \  
  --lineage mammalia_odb12
```

The first example registers a broad BUSCO lineage. The second registers a more specific lineage and records that it sits under the broader metazoan library context.

8.2.2 Running BUSCO on assemblies

`busco-run` analyses one accession, while `batch-busco` analyses a set. For most real projects, `batch-busco` is the practical entry point because the user generally wants BUSCO profiles for an entire candidate panel.

BUSCO runs are tracked explicitly as run records, including pipeline, mode, effective parameters, and output location. This allows multiple BUSCO runs per accession/library and later selection by run.

Pipeline-related task flags include:

- `--pipeline {auto,miniprot,metaeuk,augustus}`
- Augustus-specific:
 - `--augustus-evaluate`
 - `--augustus-limit`
 - `--augustus-long`
 - `--augustus-species`
 - `--augustus-parameters`
- Metaeuk-specific:
 - `--metaeuk-parameters`
 - `--metaeuk-rerun-parameters`
- Miniprot-specific:
 - `--miniprot-parameters`
- General:
 - `--format {auto,protein,genome,nucleotide}`
 - `--proteome-profile NAME`
 - `--isoforms-cleaned`
 - `--raw-proteome`

8.2.3 BUSCO input modes and proteome profiles

For `busco-run` and `batch-busco`, `--format` is the task's own BUSCO input mode for the run being created.

Existing-run selector flags such as `--run-ids`, `--export-format`, `--require-format`, and `--prefer-format` are not part of these task parsers; those belong to BUSCO-consuming commands like `list results`, `export`, `decontamination`, `paralog filtering`, and `set busco-primary`.

For protein-mode BUSCO, the important input choice is usually the proteome profile:

- `--raw-proteome` selects the accession's `raw` profile;

- `--isoforms-cleaned` selects the accession's current default cleaned profile, whatever its actual stored name is;
- `--proteome-profile NAME` selects an exact stored profile such as `cdhit96`, `gff`, or `gff_cdhit96`.

This allows one accession to keep several proteome variants side by side and retain separate BUSCO runs for each profile.

If you do not pass any proteome-profile selector for a protein-mode BUSCO task, PhyloODB uses:

1. the accession's stored default proteome profile;
2. otherwise the accession's default cleaned profile if one exists;
3. otherwise `raw`.

Profile creation itself is covered in [automatic proteome preparation after download](#). The main BUSCO rule is simpler: choose `--raw-proteome`, `--isoforms-cleaned`, or `--proteome-profile NAME` when the BUSCO run must use a specific proteome representation, and omit them when the accession default is the desired input.

BUSCO run examples

```

phyloODB my_project.db queue busco-run \
  --accession GCF_000001405.40 \
  --lineage metazoa_odb12 \
  --format genome \
  --pipeline augustus \
  --augustus-species human \
  --augustus-limit 5

phyloODB my_project.db queue batch-busco \
  --accessions @MAMMAL_TARGETS \
  --lineage mammalia_odb12 \
  --format genome \
  --pipeline metaeuk \
  --metaeuk-parameters "--max-overlap=15,--max-intron=200000"

phyloODB my_project.db queue batch-busco \
  --accessions @METAZOA_CANDIDATES \
  --lineage metazoa_odb12 \
  --format protein

phyloODB my_project.db queue batch-busco \
  --accessions @METAZOA_CANDIDATES \
  --lineage metazoa_odb12 \
  --format protein \
  --isoforms-cleaned

phyloODB my_project.db queue batch-busco \
  --accessions @METAZOA_CANDIDATES \
  --lineage metazoa_odb12 \
  --format protein \
  --proteome-profile gff_cdhit99

```

When BUSCO format or pipeline options are omitted, the project defaults `DEFAULT_BUSCO_FORMAT` and `DEFAULT_BUSCO_PIPELINE` are used where the task needs to choose automatically.

Genome-mode pipeline choices

For genome-mode eukaryote runs, the practical distinction is:

- `miniprot` : protein-oriented output (`aa-only`) and fast screening; `miniprot ref.mpi` cleanup is controlled by `BUSCO_MINIPROT_KEEP_REF_FILE` .
- `augustus` and `metaeuk` : support nucleotide-oriented downstream export (`aa+nt`).

Note: nucleotide export is only available from suitable run types (`augustus` / `metaeuk`).

This is not a superficial distinction. It affects what can be exported later.

- a `protein` BUSCO run is suitable for protein-oriented filtering and reporting;
- a `genome` BUSCO run from `metaeuk` or `augustus` is what enables nucleotide-family export downstream;
- a project may therefore intentionally keep both a protein-oriented run and a genome-oriented run for the same accession.

This is why PhyloODB tracks BUSCO runs explicitly rather than flattening them into one score row per accession.

8.2.4 OrthoFinder runs

`orthofinder-run` is used in the construction of derived libraries rather than in routine screening. It provides the orthology context required by `add-library` .

`orthofinder-run` can also take an explicit MCL inflation override via `--mcl-inflation` . This value is stored with the OrthoFinder run record and is part of run identity for reuse. In practical terms, the same accession set with a different inflation value is treated as a different OrthoFinder analysis rather than as a cache hit.

```
phyloODB my_project.db queue orthofinder-run \  
  --accessions @METAZOA_REFERENCES \  
  --library-name metazoa_odb12 \  
  --proteome-profile gff  
  
phyloODB my_project.db queue orthofinder-run \  
  --accessions @METAZOA_REFERENCES \  
  --library-name metazoa_odb12 \  
  --proteome-profile gff \  
  --mcl-inflation 1.8
```

When PhyloODB discovers an older OrthoFinder folder and considers ingesting it for reuse, it reads `Log.txt` and inspects the `Command Line:` entry. If that command includes `-I` , the parsed inflation value is stored on the run and compared against the current request. If no `-I` is present, the run is treated as using OrthoFinder's default inflation setting.

8.2.5 Derived library construction

`add-library` is one of the central tasks of PhyloODB. It creates a derived orthology library from a parent BUSCO lineage and a reference set of accessions. Conceptually, it asks which BUSCO families in the parent lineage are sufficiently consistent with OrthoFinder-defined orthogroups across the reference panel to be retained as a conservative, study-specific library.

This is the step that turns generic BUSCO lineages into project-specific core sets.

```
phyloODB my_project.db queue add-library \  
  --name metazoa_core \  
  --parent-library-name metazoa_odb12 \  
  --coverage Metazoa \  
  --accessions @METAZOA_REFERENCES  
  
phyloODB my_project.db queue add-library \  
  --name metazoa_core \  
  --parent-library-name metazoa_odb12 \  
  --coverage Metazoa \  
  --accessions @METAZOA_REFERENCES \  
  --force \  
  --rerun-gene-trees \  
  --annotate-og-trees
```

The first command builds a derived library from reusable upstream BUSCO and OrthoFinder evidence where possible. The second rebuilds the derived library, recomputes replacement gene trees, and writes annotated copies for inspection, while still reusing BUSCO and OrthoFinder runs unless their own rerun flags are also supplied.

Two core-set strategies are now available:

- default paralog-aware mode: accept exact 1:1 BUSCO/orthogroup matches only after the selected gene-tree workflow has been used to classify paralogs;
- `--skip-paralog-analysis` : stop after exact 1:1 BUSCO/orthogroup definition plus occupancy filtering and accept those families directly.

When paralog-aware mode is used, the gene-tree source can also be chosen:

- default `--gene-tree-source iqtree` : build replacement MAFFT alignments and IQ-TREE gene trees, then write the canonical core-set trees under `IQ-TREE_Orthogroup_trees` ;

- `--gene-tree-source fasttree` or `--fast-tree` : reuse OrthoFinder Resolved_Gene_Trees directly and skip both replacement alignment and IQ-TREE work;
- `--annotate-og-trees` : queue `annotate-orthogroup-tree` after the library gene trees are available, so the inspected tree files include OrthoFinder, BUSCO, and paralogy context.

Rebuild and rerun controls

Operationally, `add-library` now distinguishes between three different kinds of refresh:

- rebuild the derived library record and its own output directory;
- rerun BUSCO on the reference accessions;
- rerun OrthoFinder on the reference accession set.

These are no longer treated as the same thing.

- `--force` allows an existing derived library to be rebuilt in place. It purges the stored derived-library state for that library and removes the derived library output directory before reconstruction.
- `--force` does not by itself rerun BUSCO or OrthoFinder.
- `--rerun-busco` is the explicit request to regenerate BUSCO runs for the references.
- `--rerun-orthofinder` is the explicit request to regenerate OrthoFinder results for the reference set.
- `--orthofinder-mcl-inflation` is the explicit request to run the child OrthoFinder stage with a non-default MCL inflation value.
- `--rerun-gene-trees` is the explicit request to regenerate replacement IQ-TREE orthogroup trees even when reusable resolved trees are already present.
- `--skip-paralog-analysis` is the explicit request to accept exact 1:1 families without tree-based paralog classification.
- `--gene-tree-source fasttree` is the explicit request to reuse OrthoFinder FastTree trees instead of building replacement IQ-TREE trees.

This matters because OrthoFinder can be expensive, and in many operator workflows the correct action is “rebuild the library from the current valid BUSCO and OrthoFinder evidence” rather than “recompute everything from scratch”.

In practice:

- `--force` alone means “rebuild the derived library and recompute its core-set analysis from reusable upstream results if they are still valid”;
- `--force --rerun-busco` means “rebuild the library and replace the BUSCO evidence”;
- `--force --rerun-orthofinder` means “rebuild the library and replace the OrthoFinder evidence”;
- `--force --rerun-gene-trees` means “rebuild the library and recompute replacement orthogroup trees even if previous resolved IQ-TREE trees are available”;
- `--force --gene-tree-source fasttree` means “rebuild the library using OrthoFinder FastTree trees without replacement alignment or IQ-TREE work”;
- `--force --skip-paralog-analysis` means “rebuild the library from exact 1:1 BUSCO/orthogroup matches only”;
- `--force --rerun-busco --rerun-orthofinder` is the full from-scratch rebuild.

When `--force` is used without `--rerun-orthofinder`, an existing OrthoFinder run already linked to that derived library is preserved and may be reused. When `--rerun-orthofinder` is supplied, that library-linked OrthoFinder result is discarded as part of the rebuild expectation.

Each derived library writes `library_build_metadata.json` in its library root. This records the effective core-set strategy, tree source, accepted-family rule, and the main analysis output paths so the library definition is explicit later.

When replacement IQ-TREE gene trees are part of that workflow, the library analysis output also records the main tree products needed for inspection:

- replacement alignments in the library analysis area;
- per-orthogroup IQ-TREE result directories;
- `orthogroup_tree_manifest.tsv`, which links each orthogroup to the alignment and resolved tree path used downstream;
- annotated orthogroup trees under `annotated-og-trees/` when `--annotate-og-trees` was requested.

Reference BUSCO cleaning

`add-library` also supports OrthoFinder-derived cleaning of the reference BUSCO runs:

- `--clean-refs` : duplicate a BUSCO family only when that accession is specifically found in an out-paralog set in the mapped exact 1:1 orthogroup;
- `--clean-refs-strict` : duplicate a BUSCO family when that accession is specifically found in either an in-paralog or out-paralog set in the mapped exact 1:1 orthogroup;
- `--set-cleaned-primary` : make the derived `pipeline=orthofinder` BUSCO run the default primary run for those references;
- `--no-set-cleaned-primary` : create the cleaned runs without changing BUSCO primaries.

This produces a new BUSCO-run record for the reference assemblies under the OrthoFinder (O) pipeline.

The key design point is that these cleaned BUSCO runs are accession-specific. A family is not reclassified merely because an orthogroup is globally labelled as containing paralogs. The reclassification only happens when the tree evidence shows that the accession itself participates in the relevant in-paralog or out-paralog tuple.

When `--skip-paralog-analysis` is used, these cleaned reference BUSCO rewrites are not created because there is no accession-specific paralog evidence to apply.

Internal alignment and tree tasks

The lower-level `mafft-run` and `iqtree-run` tasks expose the two tree-building steps independently. `add-library` uses this machinery when it builds replacement IQ-TREE orthogroup trees. They are also useful when you already have a FASTA or alignment and want PhyloODB to record the task and artifacts in the database.

```

phyloODB my_project.db queue mafft-run \
  --input-fasta libraries/metazoa_core/analysis/orthogroups/OG0001234.faa \
  --out-dir libraries/metazoa_core/analysis/alignments

phyloODB my_project.db queue iqtree-run \
  --input-alignment
libraries/metazoa_core/analysis/alignments/OG0001234.aln.fasta \
  --out-dir libraries/metazoa_core/IQ-TREE_Orthogroup_trees \
  --prefix OG0001234

```

`mafft-run` uses `MAFFT_PATH` and `MAFFT_FLAGS` when set, otherwise it looks for `mafft` on `PATH` and uses a conservative default alignment recipe. `iqtree-run` similarly uses `IQTREE_PATH` and `IQTREE_FLAGS`, falling back to `iqtree2` or `iqtree` on `PATH`.

Tree annotation

`annotate-orthogroup-tree` annotates one tree, or a directory of trees, using OrthoFinder and BUSCO/paralogy metadata. It is mainly an inspection tool for understanding orthogroup trees after [library construction](#) or after separate tree-building work.

In normal derived-library work, tree annotation is part of the `add-library` workflow when `--annotate-og-trees` is supplied. The annotation subtask writes into the derived library directory, under the [active libraries root](#), using the path:

```
<libraries-root>/<library-name>/annotated-og-trees/
```

For example, a custom library called `metazoa_core` stored under the default libraries root would have annotated trees in `libraries/metazoa_core/annotated-og-trees/`. The sibling `library_build_metadata.json` records whether annotation was requested and effective for that build.

```
phyloODB my_project.db queue annotate-orthogroup-tree \  
  --input-tree libraries/metazoa_core/IQ-  
TREE_Orthogroup_trees/OG0001234.treefile \  
  --orthofinder-location orthofinder/Results_MetazoaCore \  
  --out-dir libraries/metazoa_core/annotated-og-trees  
  
phyloODB my_project.db queue annotate-orthogroup-tree \  
  --input-dir libraries/metazoa_core/IQ-TREE_Orthogroup_trees \  
  --manifest-tsv libraries/metazoa_core/orthogroup_tree_manifest.tsv \  
  --out-dir libraries/metazoa_core/annotated-og-trees
```

Use `--input-tree` for one tree or `--input-dir` for a folder, not both. When available, `--manifest-tsv`, `--mapping-tsv`, `--species-paralog-tsv`, `--orthofinder-location`, and `--source-run-ids` give the annotator more context about which orthogroups, BUSCO families, and paralog calls the tree represents.

8.2.6 Custom library import

`import-custom-library` is a lower-level alternative for cases where the family list has already been decided externally and simply needs to be registered. It records a custom BUSCO-family set under a new library name, attaches a coverage taxid or label, links the new library to a parent lineage, and optionally records reference accessions.

8.3 Hidden paralog filtering

Relevant tasks:

- `create-proteome-blast-db`
- `construct-busco-blast-db`
- `paralog-removal`
- `paralog-filtering`
- `filter-paralogs`

PhyloODB assumes that a custom library alone may not be sufficient to exclude hidden paralogs. BUSCO assignment is heuristic, and a sequence can still satisfy BUSCO's model while corresponding to the wrong copy within a family.

`create-proteome-blast-db` and `construct-busco-blast-db` are support tasks for BLAST-backed screening. They are normally invoked indirectly by higher-level workflows, but they can be useful when building or debugging reusable BLAST resources explicitly.

`create-proteome-blast-db` creates a BLAST database from one accession's protein set. It can use the accession's current default proteome profile, a named profile, a preferred profile, the default cleaned profile, or the raw proteome.

```
phyloODB my_project.db queue create-proteome-blast-db \  
  --accession GCF_000001405.40 \  
  --library-name metazoa_core \  
  --proteome-profile gff_cdhit96
```

`construct-busco-blast-db` creates a BLAST database from BUSCO sequences for a selected accession set. It can be scoped to a BUSCO library, restricted to particular family ids, and optionally built from paralog-filtered BUSCO calls.

```
phyloODB my_project.db queue construct-busco-blast-db \  
  --accessions @TARGETS \  
  --busco-library-id 1 \  
  --target-library-id 2 \  
  --use-paralog-filtered \  
  --output-path cache/target_bscos
```

`paralog-removal` addresses this by comparing BUSCO-derived sequences from the target accessions against whole reference proteomes. At a high level, the reference panel defines what the expected homologous copy should be. If a target sequence consistently matches a different copy better than the expected one, the BUSCO family can be marked as unsafe for that accession. The comparison remains whole-proteome on purpose: hidden paralogs are often not the BUSCO-labelled ortholog itself. It is best to use Reciprocally Validated (O pipeline) results for the reference panel.

When BUSCO selector flags are used with `paralog-removal`, they define which BUSCO run is used to supply the target and reference families. This means:

- `--busco-pipeline augustus` or `--require-busco-pipeline augustus` requires Augustus runs;
- but the task still obeys `--busco-run-selection` ;
- so `--busco-pipeline augustus` by itself means “use the Augustus primary run if one exists”, not “use any Augustus run”.

For most operator use, if the intention is “run paralog filtering against Augustus-derived BUSCO families wherever Augustus results exist”, the safer form is:

```
phyloODB project.db queue paralog-removal \  
  --library-name ecdysozoa_core \  
  --accessions @TARGETS \  
  --busco-pipeline augustus \  
  --busco-run-selection latest
```

If you instead want to restrict the task to accessions whose current primary BUSCO run is already Augustus, keep `--busco-run-selection primary`.

The result is not a new library but a set of accession-specific filtering decisions. This distinction matters. A custom library defines which families exist in principle for the study. Paralog filtering decides whether a particular accession contributes a trustworthy member of a family.

Selection of reference proteomes is now explicit. `--mode median` remains the default and reproduces the previous behaviour. Alternative modes are available:

- `--mode lower-quartile` : use the lower quartile threshold per family, which usually compares against more reference proteomes.

- `--mode upper-quartile` : use the upper quartile threshold per family, which is stricter.
- `--mode percent --percentile N` : rank reference proteomes by BUSCO bitscore for each family and compare against the top N% .
- `--mode bitscore --bitscore-threshold X` : compare against references above a fixed raw bitscore.

Reports are written under the shared reports root by default, seeded from `REPORTS_DIR` and managed as the active `reports` storage root. For paralog filtering this means a directory such as `paralog-filtering-reports/task_<task_id>_<timestamp>_<label>/` containing:

- a per-accession summary;
- a per-family selection report listing how many reference proteomes were selected;
- per-decision rows showing clean/dirty calls and reuse;
- per-accession BLAST hit reports under `blast_hits/` , written after each accession finishes and showing the evidence used for each query/reference comparison.

`--report-dir` can be used to write those report files to an explicit directory instead. The older `--out-dir` option is no longer used for paralog filtering.

8.4 Decontamination

[[[SECTION UNDER CONSTRUCTION]]]

Relevant tasks:

- `decontamination`
- `internal-decontamination`
- `external-decontamination-check`
- `external-decontamination-apply`

PhyloODB currently supports two broad decontamination strategies.

`decontamination` is reference-based. It BLASTs BUSCO-derived sequences from target assemblies against a chosen reference set and judges whether the best-supported hits fall inside or outside the expected taxonomic range.

`internal-decontamination` uses the target set itself. It builds an internal BUSCO BLAST database, examines hit enrichment within the target set, and can optionally call an external BLAST confirmation stage. This is especially useful when one wants to ask whether a target assembly behaves as a coherent member of the sampled clade before bringing in broader reference resources.

Both routes write per-BUSCO and per-accession decisions into the database under explicit run identifiers. This is important because decontamination is not always a once-and-for-all judgement; one may repeat it with different reference panels or thresholds.

`external-decontamination-check` is the optional confirmation stage for internal decontamination. It takes BUSCOs flagged by an internal run and checks them against an external BLAST database, such as a local copy of `nr`, so that suspicious internal signals can be tested against a broader reference space.

`external-decontamination-apply` takes the external BLAST output and applies it back to a previous internal decontamination run, writing a new run id rather than mutating the original decisions. This keeps the internal-only judgement and the externally confirmed judgement separate and reproducible.

JSON configuration for decontamination

Both decontamination routes accept `--config-path` pointing to a JSON file. This is the best way to document a complex screen because the thresholds, targets, and acceptance rules become explicit and reusable.

For reference-based `decontamination`, the current implementation accepts a JSON object with:

- `params` : threshold and selector overrides;
- `targets` : optional explicit targets;
- `references` : optional explicit references;
- `groups` or `acceptance_rules` : optional rule blocks.

A practical example is:


```

{
  "params": {
    "rank": "order",
    "off_clade_fraction": 0.10,
    "min_buscos": 20,
    "min_identity": 70,
    "min_coverage": 70,
    "min_delta_bitscore": 20,
    "min_hits": 1,
    "hit_window": 1,
    "ref_clade": "Mammalia",
    "ref_rule_rank": "order",
    "ref_rule_quantity": 1,
    "allow_same_species": false
  },
  "targets": [
    "GCF_000001405.40",
    "GCF_037993035.2",
    "GCF_049350105.2"
  ],
  "references": [
    "Primates",
    "Rodentia",
    "Carnivora"
  ],
  "groups": [
    {
      "members": ["Homo sapiens", "Pan troglodytes"],
      "clades": ["Primates"],
      "blacklist": [],
      "min_hits": 1
    },
    {
      "members": ["Macaca mulatta", "Macaca fascicularis"],
      "clades": ["Primates"],
      "blacklist": [],
      "min_hits": 1
    }
  ]
}

```

The group structure allows accession names, clade names, or taxids. The keys `members` and `targets` are both recognised inside group objects.

For `internal-decontamination`, the same `params` approach is used, but the `references` section is not central because the internal BLAST database is built from the target set itself. A useful example is:

```
{
  "params": {
    "rank": "order",
    "hit_window": 8,
    "p_value_threshold": 0.05,
    "off_clade_fraction": 0.05,
    "min_buscoss": 20,
    "external_blast_db_path": "/db/nr/nr",
    "external_blast_output_dir": "/data/internal_decontam_external"
  },
  "targets": [
    "GCF_000001405.40",
    "GCF_037993035.2",
    "GCF_049350105.2"
  ],
  "acceptance_rules": [
    {
      "members": ["Primates"],
      "clades": ["Primates"],
      "blacklist": [],
      "min_hits": 1
    }
  ]
}
```

The JSON layer is especially valuable when the screening logic itself, rather than merely the accession list, needs to be preserved as part of the study design.

8.5 Export and reporting

Relevant tasks:

- `export`
- `build-busco-trees`
- `generate-lineage-csv`

`export` is the endpoint of most projects. It uses the chosen library together with the current filtering context to write per-family FASTA datasets and associated reports.

This behaviour should be understood clearly. Export is not merely “write all BUSCOs”. It is an assembly step that can enforce the study’s current quality policy. Depending on flags, it can require prior filtering, ignore filtering, or target a specific decontamination run.

`generate-lineage-csv` is a lightweight reporting task. It resolves selectors to accessions and writes their lineage information to CSV without exporting BUSCO family FASTAs. It is useful when you need a taxon table for inspection, publication supplements, or downstream scripts.

```
phyloODB my_project.db queue generate-lineage-csv \  
  --accessions @TARGET_PANEL \  
  --output reports/target_lineage.csv
```

8.5.1 `export` : `--require` , `--header` , and related settings

Several export options deserve explicit treatment because they strongly affect the structure of the final dataset.

`--require` is a family-presence filter over the selected taxon set. Conceptually, it says that a family should be retained only if it contains at least one accession from each required clause.

- clauses are ANDed together;
- inside a clause, alternatives can be separated with `|` ;
- clade names and taxids are both accepted;
- exact accessions are accepted when prefixed as `acc:ACCESSION` or

```
accession:ACCESSION .
```

Examples:

```
phyloODB my_project.db queue export \  
  --library-name metazoa_core \  
  --accessions @MY_PANEL \  
  --require Primates Rodentia
```

```
phyloODB my_project.db queue export \  
  --library-name metazoa_core \  
  --accessions @MY_PANEL \  
  --require '(Primates|Glires)' Carnivora
```

```
phyloODB my_project.db queue export \  
  --library-name metazoa_core \  
  --accessions @MY_PANEL \  
  --require '(acc:GCA_000001405.29|Primates)' Carnivora
```

The first says that every retained family must include at least one primate and at least one rodent. The second says that every retained family must include at least one accession from either Primates or Glires, and also at least one from Carnivora.

`--header` controls how FASTA headers are rewritten during export, unless `--retain-headers` is set. The current implementation accepts the following tokens:

- `ACCESSION`
- `TAXON`
- `KINGDOM`
- `PHYLUM`
- `CLASS`
- `ORDER`
- `FAMILY`
- `GENUS`
- `SPECIES`
- `RANK`
- `BUSCO`
- `LENGTH`
- `GENE`

- TAXID
- BITSORE

Only the following separator characters are accepted between tokens:

- .
- |
- _
- -
- :
- [and]

Examples:

```
phyloODB my_project.db queue export \
  --library-name metazoa_core \
  --accessions @MY_PANEL \
  --header 'BUSCO:TAXON:RANK' \
  --header-rank phylum

phyloODB my_project.db queue export \
  --library-name metazoa_core \
  --accessions @MY_PANEL \
  --header 'ACCESSION_SPECIES_GENE'
```

Here RANK is the generic rank token driven by --header-rank, while fixed rank tokens such as PHYLUM and ORDER come directly from the accession lineage. FAMILY means the taxonomic family name, and BUSCO means the BUSCO family id.

--header-rank only matters if RANK is used in the template. If --retain-headers is present, PhyloODB keeps the original BUSCO headers and skips template rendering.

Other export controls that should be understood together are:

- --out-dir : optional explicit export directory. If omitted, PhyloODB creates a default task-named directory under the active exports root, initially seeded from EXPORTS_DIR .
- --disable-paralog-filter and --disable-decont-filter : ignore those filters during export, even if results exist.
- --require-paralog-filtering and --require-decontamination : insist that

those analyses exist for the selected accessions and fail export if they do not.

- `--min-completeness` and `--min-single-copy-complete` : BUSCO selector thresholds applied before export.
- `--min-occupancy` : minimum family occupancy across the selected taxa.
- `--min-taxa-occupancy` : minimum proportion of retained families each accession must participate in.
- `--sequence-type {protein,nucleotide}` : choose exported sequence type.
- `--busco-pipeline` / `--require-busco-pipeline` , `--prefer-busco-pipeline` , `--format` / `--require-format` , `--prefer-format` , and `--busco-run-selection` : resolve which BUSCO run context should drive export and filtering.
- `--write-lineage-csv` , `--write-busco-report` , `--write-busco-family-matrix` : control the auxiliary reports written beside the FASTA output.
- `--busco-report-extended` : add more decontamination detail to the BUSCO report.

In this context `--format` still means BUSCO input format, not export output format. Export output format is controlled separately by `--sequence-type` .

Export reports now live with the export itself. If `--out-dir` is set, the lineage CSV, BUSCO report, taxa-occupancy TSV, BUSCO family matrix, filter report, export task log copy, and parameter summary are written there unless a more specific explicit report path is provided. If `--out-dir` is omitted, the same files are written to the automatically created directory under the active `exports` root.

The filtering rule is important enough to state explicitly:

- by default, export uses paralog-filtering and decontamination results when they are available for the chosen library and selected accessions;
- by default, export does not fail merely because one or both filtering analyses have not been run yet;
- `--require-paralog-filtering` and `--require-decontamination` turn missing filtering state into an explicit error;
- `--disable-paralog-filter` and `--disable-decont-filter` tell export to ignore those filtering results even if they are present.

This default keeps exploratory work easy while still allowing serious dataset exports to insist on completed filtering.

8.5.2 What export writes

Every export directory contains a `busco_families/` directory with one FASTA per retained BUSCO family after export-side filtering and occupancy thresholds have been applied.

The same export directory can also contain:

- `lineage.csv` : accession-to-lineage table for the retained taxa;
- `busco_report.tsv` : per-accession BUSCO summary in the export context;
- `taxa_occupancy.tsv` : which taxa survived the minimum taxa-occupancy rule;
- `busco_family_matrix.tsv` : per-family status matrix across the retained taxa;
- `export_filter_report.tsv` : per-family removal reasons and sequence counts;
- `export_parameters.txt` : resolved export settings, selected BUSCO runs, and report locations;
- `export_task.log` : copy of the task log for that export.

These files are part of the dataset definition, not disposable side products.

When `--sequence-type nucleotide` is used, PhyloODB resolves BUSCO run choice via nucleotide-capable run mappings (augustus/metaeuk-derived runs). If no suitable run exists for selected accessions, export will report missing BUSCO run context for nucleotide export.

8.5.3 Export-aligned BUSCO tree building

`build-busco-trees` is an export-oriented tree-building command. It starts from exported BUSCO family FASTAs rather than from OrthoFinder orthogroups.

`build-busco-trees` writes:

- `busco_families/` : the export-stage FASTAs used as tree inputs;
- `alignments/` : one MAFFT alignment per exported family;
- `trees/` : one IQ-TREE result directory per exported family;
- `manifest.tsv` : a TSV linking each family id to the raw family FASTA, alignment,

tree directory, and chosen tree file.

Use this when you want family trees that correspond exactly to a particular export rather than to the broader orthogroup analysis used during [library construction](#).

Because `build-busco-trees` is an export wrapper, it accepts the same target-selection surface as `export`. That means it can build trees directly from a selector rule, not only from a hand-written accession list.

For example, build protein BUSCO trees for one downloaded representative per primate genus:

```
phyloODB my_project.db queue build-busco-trees \  
  --library-name mammalia_core \  
  --clade Primates \  
  --rank genus \  
  --quantity 1 \  
  --sequence-type protein
```

The same works with stored accession sets or selector presets:

```
phyloODB my_project.db queue build-busco-trees \  
  --library-name metazoa_core \  
  --accessions @TARGET_PANEL \  
  --min-occupancy 0.75  
  
phyloODB my_project.db queue build-busco-trees \  
  --library-name metazoa_core \  
  --preset metazoa_tree_targets \  
  --busco-run-selection latest
```

In each case, PhyloODB first resolves the export context, writes the surviving BUSCO family FASTAs, and then queues the MAFFT and IQ-TREE subtasks for those families.

Choosing the BUSCO run for export

Export is one of the places where BUSCO run selection matters most.

If an accession has:

- a protein run; and
- a slightly lower completeness `metaeuk` genome run;

then:

- protein export may reasonably use the protein-oriented run;
- nucleotide export must use the nucleotide-capable genome-oriented run.

You should therefore be explicit when the project contains multiple BUSCO contexts.

Protein export using the primary input protein run:

```
phyloODB my_project.db run export \  
  --library-name metazoa_core \  
  --accessions @TARDIGRADES \  
  --format protein \  
  --busco-run-selection primary
```

Nucleotide export using the primary nucleotide run:

```
phyloODB my_project.db run export \  
  --library-name metazoa_core \  
  --accessions @TARDIGRADES \  
  --sequence-type nucleotide \
```

If you are deliberately testing a newer run rather than the curated primary:

```
phyloODB my_project.db queue export \  
  --library-name metazoa_core \  
  --accessions @TARDIGRADES \  
  --busco-run-selection latest
```

That distinction is often important during reruns and validation. `primary` is the curated default. `latest` is the newest matching run, which may or may not yet be the project's intended default.

8.6 Storage and diagnostic helper tasks

Relevant tasks:

- `finalize-genome-move`
- `example`

`finalize-genome-move` is normally queued by `storage move-genomes --apply`; users should not usually queue it by hand. It copies or rebinds selected genome folders to a destination root, runs verification before deleting the original source, and rolls the binding back if verification fails. It is included in the task catalogue because storage moves are tracked and resumed through the same task system as analysis work.

`example` is a demonstration no-op task, also available as `demo`. It is mainly for testing the queue, daemon, task registry, and logging machinery. It is not part of a biological workflow.

9. How tasks combine in practice to produce a dataset

A typical project proceeds roughly as follows.

1. Create the database.
2. Populate assembly metadata for the relevant clades.
3. Use selectors to inspect and refine the candidate taxon set.
4. Download chosen assemblies.
5. Prepare proteome profiles where necessary.
6. Download the required BUSCO lineage datasets.
7. Run BUSCO across the reference and target assemblies.
8. Create a derived library with `add-library` if a study-specific core set is desired.
9. Run `paralog-removal` or one of its aliases (`paralog-filtering` , `filter-paralogs`) on the target set against the chosen references.
10. Run decontamination, either reference-based or internal.
11. Export the final filtered dataset.
12. Build trees using IQTREE task or use in your own workflow.

This order reflects the logic of the system. Metadata precede download because sampling decisions depend on metadata. BUSCO precedes library construction because libraries are built from BUSCO-defined families. Paralog filtering and decontamination precede export because export should reflect filtered rather than raw BUSCO presence.

10. Suggested workflows

10.1 Exploratory survey of a clade

If the user does not yet know what to sample, the goal should be inspection rather than immediate downloading.

- Populate metadata for a broad clade.
- Use `list assemblies` and `count assemblies` with rank rules.
- Store provisional panels as variables.
- Download only a small test subset first.

This mode is particularly useful early in a project, when the user is assessing whether enough high-quality assemblies exist.

10.2 Building a conservative core library

If the objective is a custom core set for a study group, the critical step is reference panel choice.

- Choose a set of reference taxa that span the study group without being so sparse that hidden paralogs become hard to recognise.
- Ensure the parent BUSCO lineage is appropriate for the clade.
- Run BUSCO on the reference panel.
- Build the derived library with `add-library`.
- Inspect the resulting library and only then proceed to large-scale export.

The quality of the reference panel matters more than the absolute number of reference taxa. A smaller but taxonomically coherent and well-annotated panel is often better than a large collection of uneven assemblies.

10.3 Large target sampling after library construction

Once a custom library exists, target sampling can become more aggressive.

- Use selectors to choose breadth across the clade.
- Download and BUSCO-profile the target set.

- Apply paralog filtering and decontamination.
- Export with occupancy thresholds appropriate to the question.

This is the phase in which PhyloODB's database-centred design is most valuable, because the target set can be revised repeatedly while the library and prior results remain available.

11. Interpreting outputs

The most important point is that PhyloODB distinguishes between several kinds of evidence.

- Library membership: whether a BUSCO family belongs to the derived study library.
- BUSCO presence: whether a target accession contains that family according to BUSCO.
- Paralog cleanliness: whether the BUSCO-derived sequence appears to be the expected copy.
- Decontamination status: whether the sequence behaves as expected taxonomically.
- Export eligibility: whether the family and accession satisfy the current export rules.

Users should avoid collapsing these into one question such as “is the gene present?”. In PhyloODB, a family may be present according to BUSCO yet excluded from export because it failed paralog filtering or decontamination. This is a strength of the system, not a nuisance.

`export` can also produce supporting reports such as lineage CSVs, BUSCO reports, and BUSCO family matrices. These should be treated as part of the dataset definition, because they document which taxa and loci ultimately survived the filtering pipeline.

12. Database management and recovery

This section is about running an existing PhyloODB project as an operator rather than as a one-shot pipeline user. In practice, this means understanding four things:

- where the program thinks data will live;
- how to move those data without breaking bindings;
- how to remove stale records or stale files safely;
- how to rediscover or re-verify data when the filesystem and database need reconciling.

Database schema upgrades are explicit:

```
phyloODB my_project.db migrate
```

Normal commands never modify the schema automatically. If an older database needs upgrading, PhyloODB reports its version and the migration command to run.

12.1 Storage roots: what they are and how they work

Storage roots are the registered base directories for major categories of project data. A root is not an individual assembly folder. It is the parent directory under which many objects may live.

Typical root kinds are:

- genomes
- libraries
- orthofinder
- exports
- reports
- logs
- cache
- misc

The usual default layout for a new database is a set of sibling folders in the same directory as the database file:

```
project/
├── project.db
├── genomes/
├── libraries/
├── orthofinder/
├── exports/
├── reports/
├── logs/
├── cache/
└── misc/
```

You can inspect the registered roots with:

```
phyloODB project.db list roots
```

The initial roots are seeded from variables such as `GENOME_DIR` , `LIBRARIES_DIR` , `ORTHOFINDER_OUTPUT_DIR` , `EXPORTS_DIR` , `REPORTS_DIR` , `LOG_DIR` , `CACHE_DIR` , and `MISC_DIR` . After creation, registered storage roots are the authoritative model; changing a path variable is not a substitute for `storage add-root` , `storage activate-root` , or `storage rebind-root` .

Task and daemon logs use the active `logs` root by default, writing to `phyloodb.log` under that root. Log behaviour can also be tuned with `LOG_LEVEL` , `LOG_TO_CONSOLE` , `LOG_MAX_BYTES` , and `LOG_BACKUPS` . A command-line daemon `--logfile` override still

wins for that daemon invocation. Older databases that only have `LOG_FILE` can be backfilled into the root model from the parent directory of that file.

or equivalently:

```
phyloODB project.db storage roots
```

For `genomes` , `libraries` , `orthofinder` , and `exports` , PhyloODB uses a single-active-root.

- only one root of that kind is active at a time;
- the active root is the write target for new data;

- inactive roots remain valid for existing data already bound there.

For example, it is entirely valid to have:

- an SSD `genomes` root as the active write target for new downloads;
- an HDD `genomes` root holding older assemblies;
- BUSCO and verification continuing to work on both.

In this model, inactive does not mean inaccessible. It means “not chosen automatically for new writes”.

12.2 Active and inactive roots in real work

The usual workflow is:

1. add a new root;
2. move data into it if desired;
3. activate it only when you want future writes to land there.

Add a second genomes root:

```
phyloODB project.db storage add-root \  
  --kind genomes \  
  --base-path /mnt/hdd/phyloodb/genomes \  
  --label HDD_GENOMES
```

For non-first roots of the strict working kinds, PhyloODB creates them inactive by default. That is deliberate. Adding a new drive should not silently redirect future downloads.

Activate that root later:

```
phyloODB project.db storage activate-root HDD_GENOMES
```

Deactivate a root without selecting a replacement:

```
phyloODB project.db storage deactivate-root HDD_GENOMES
```

Labels must match exactly, and storage-root labels are required to be unique. PhyloODB rejects attempts to create a second root with an existing label.

Root base paths must also be unique and non-overlapping. PhyloODB rejects a root whose base path is identical to, inside, or containing another registered root base path.

This is allowed, but it has a real operational consequence. If no active root remains for `genomes`, `libraries`, `orthofinder`, or `exports`, then any task that needs to create new data of that kind is blocked until a root is activated again.

12.3 Moving a whole root versus moving selected data

There are two different move problems, and they should not be treated as the same.

Whole-root move

Use this when the entire base path has changed but the relative layout underneath it is still the same.

Example:

- before: `/ssd/project/genomes/ACC1`
- after: `/hdd/project/genomes/ACC1`

In that case the root itself has moved. The correct command is:

```
phyloODB project.db storage rebind-root 1 \  
  --base-path /hdd/project/genomes
```

or, using a label:

```
phyloODB project.db storage rebind-root SSD_GENOMES \  
  --base-path /hdd/project/genomes
```

This is dry-run by default. Re-run with `--apply` when the preview looks correct:

```
phyloODB project.db storage rebind-root 1 \  
  --base-path /hdd/project/genomes \  
  --apply
```

This updates the base path of the root and queues targeted verification by default. It is the right choice when you have already moved the directory tree outside PhyloODB, for example with `rsync`, `mv`, or a filesystem snapshot promotion.

Selected-data move

Use this when only some genomes or some libraries are moving to a different root.

For genomes:

```
phyloODB project.db storage move-genomes \  
  --accessions @OLD_GENOMES \  
  --to-root HDD_GENOMES
```

For libraries:

```
phyloODB project.db storage move-libraries \  
  --library-name metazoa_core \  
  --to-root HDD_LIBRARIES
```

Again, these commands are dry-run by default. The preview shows exactly:

- source path;
- destination path;
- source root id;
- destination root id;
- action.

To perform the move:

```
phyloODB project.db storage move-genomes \  
  --accessions @OLD_GENOMES \  
  --to-root HDD_LIBRARIES \  
  --apply
```

For genome moves, `--apply` queues a genome-move finalization task rather than deleting the source immediately.

With the default `--verify` behavior, that task does this:

1. copy the accession directory to the destination root;
2. update the genome binding to point at the copied destination;
3. queue `verify-assembly --repair --tidy` first, then `verify-busco --repair --reingest` ;
4. suspend until both child tasks complete;
5. if verification succeeds, delete the original source directory;
6. if verification fails, roll the genome binding back to the original source and remove the copied destination.

This is safer than a direct filesystem move because the original directory is not deleted until the destination has passed verification.

Use `--rebind-only` only when you have already moved the directories manually and now want the database to catch up.

```
phyloODB project.db storage move-genomes \  
  --accessions @OLD_GENOMES \  
  --to-root HDD_LIBRARIES \  
  --rebind-only \  
  --apply
```

12.4 What verification happens after moves

Move and rebind commands queue verification by default. This is important because moving paths is not just about the top-level folder location. The database may also need to reconcile artifact bindings and normalized assembly files.

After genome moves and genome root rebinds, PhyloODB queues:

```
verify-assembly --repair --tidy  
verify-busco --repair --reingest
```

That means the follow-up verification can also normalize expected compressed forms such as:

- `.fna.gz`
- `.faa.gz`
- `.faa.archive.gz`

- GFF compression and normalization where applicable

The BUSCO verification step matters because BUSCO run records and BUSCO artifacts are separate from the top-level genome binding. After a genome move, `verify-busco --repair --reingest` is what re-registers BUSCO result locations cleanly and rewrites the run identity against the moved on-disk result so downstream export resolves the moved BUSCO directories without falling back to legacy guesses.

After library moves and library root rebinds, PhyloODB queues:

```
verify-libraries --repair
```

12.5 Practical SSD/HDD scenarios

Scenario 1: keep new downloads on SSD, archive older genomes to HDD

This is the most common arrangement.

1. The SSD genomes root is active.
2. Add an HDD genomes root.
3. Move older accessions there.
4. Leave the HDD root inactive.

Commands:

```
phyloODB project.db list roots

phyloODB project.db storage add-root \
  --kind genomes \
  --base-path /mnt/hdd/phyloodb/genomes \
  --label HDD_GENOMES

phyloODB project.db storage move-genomes \
  --accessions @ARCHIVE_SET \
  --to-root HDD_GENOMES

phyloODB project.db storage move-genomes \
  --accessions @ARCHIVE_SET \
  --to-root HDD_GENOMES \
  --apply
```

Result:

- archived genomes now live on the HDD;
- they remain fully usable for BUSCO, verification, listing, and export;
- new downloads still land on the SSD because the SSD root remains the active genomes root.

You can also scope verification directly to a root instead of first materialising an accession set:

```
phyloODB project.db queue verify-assembly --root HDD_GENOMES --repair --tidy
phyloODB project.db queue verify-busco --root HDD_GENOMES --repair
```

For library-scoped verification, `--root` refers to a libraries root:

```
phyloODB project.db queue verify-libraries --root HDD_LIBRARIES --repair
```

For OrthoFinder verification, `--root` refers to an orthofinder root:

```
phyloODB project.db queue verify-orthofinder --root OF_HDD --repair
```

Scenario 2: switch the active working root because the SSD is full

Once the new drive should become the main working location:

```
phyloODB project.db storage activate-root HDD_GENOMES
```

From that point onward, new assembly downloads use that root. Existing genomes on older roots remain valid.

Scenario 3: suspend new downloads temporarily

If you want to prevent new genome-producing tasks from starting until storage is reorganized:

```
phyloODB project.db storage deactivate-root HDD_GENOMES
```

This leaves the project readable but suspends new genome writes until a root is activated again.

12.6 The artifact system in depth

The artifact system is how PhyloODB keeps track of derived files beyond the primary “this assembly lives in this folder” or “this library lives in this folder” bindings.

Conceptually, an artifact is:

- owned by something;
- typed;
- stored under a root plus relative path when possible;
- recoverable even if its base root moves.

This is why root rebinding matters. If an artifact is stored as:

- `storage_root_id = 4`
- `relative_path = busco/run_1/short_summary.json`

then changing the base path of root `4` changes the resolved artifact location automatically without rewriting every artifact row to a fresh absolute path.

12.7 Purging records, files, settings, and roots

`purge` is for selective cleanup. It is dry-run by default and should stay that way until you have reviewed the preview carefully.

Current purge subjects include:

- `assemblies`
- `decontamination`
- `busco`
- `hidden-paralog`
- `libraries`
- `variables`
- `roots`

Purging custom variables or settings

To remove only custom variables while keeping system configuration:

```
phyloODB project.db purge variables --custom-only
```

Apply:

```
phyloODB project.db purge variables --custom-only --apply
```

This is useful when stored accession sets or ad hoc flags have accumulated and you want to clean the configuration layer without damaging the project itself.

Purging assemblies and optionally deleting files

To remove assembly-linked rows:

```
phyloODB project.db purge assemblies --accessions GCF_000001405.40
```

To also delete the referenced files:

```
phyloODB project.db purge assemblies \  
  --accessions GCF_000001405.40 \  
  --delete-files \  
  --apply
```

This is intentionally constrained to known data roots. The purge system does not blindly delete arbitrary filesystem paths.

Purging libraries or BUSCO/decontamination-derived state

Examples:

```
phyloODB project.db purge libraries --library-name metazoa_core  
phyloODB project.db purge busco --library-name metazoa_odb12 --accessions  
@BAD_SET  
phyloODB project.db purge decontamination --run-id idc_abcdef123456
```

These are recovery tools. Use them when the derived state is wrong and you intend to recompute or rediscover it.

Purging roots

Roots can also be purged:

```
phyloODB project.db purge roots --inactive-only
```

or:

```
phyloODB project.db purge roots --root-id 7
```

Rules:

- dry-run by default;
- active roots are protected unless explicitly forced;
- bound roots are blocked from deletion;
- if purging a strict working root would leave that kind without an active root, PhyloODB warns that new writes for that kind are suspended until another root is activated.

The general rule is that root purging is for cleaning unused storage metadata, not for deleting live project data.

12.8 Rediscovery and filesystem reconciliation

Sometimes the filesystem changes outside PhyloODB. Examples include:

- you inherit a project from someone else;
- directories were moved manually;
- BUSCO results exist on disk but were never fully ingested;
- an interrupted migration left the database lagging behind the files.

This is what rediscovery and verify are for.

```
phyloODB project.db discover
```

With flags:

- `--root <id|label>`

- `--path <path>`
- `--dry-run`
- `--overwrite`
- `--attempt-knowledge-update`

Interpretation:

- `discover` scans registered genomes roots and ingests assemblies and BUSCO runs it finds;
- `plain discover` scans all registered genomes roots, including inactive ones;
- `discover --root <ROOT>` scans one registered genomes root;
- `discover --path <PATH>` scans one subtree inside a registered genomes root;
- `--dry-run` previews what would be discovered or rebound;
- `--overwrite` allows known accessions found at a different path to be rebound to the discovered location.

Important safety rule:

- `--path` must already be inside a registered genomes root;
- if it is not, discovery errors and tells you to register that root first with `storage add-root --kind genomes --base-path ...`.

Discovery does not create storage roots for you and it does not silently bind out-of-root paths to the current active root.

Examples:

```
phyloODB project.db discover
phyloODB project.db discover --root HDD_GENOMES
phyloODB project.db discover --path /data/project/genomes/subset_a
phyloODB project.db discover --root HDD_GENOMES --dry-run
phyloODB project.db discover --root HDD_GENOMES --overwrite
```

This is especially useful when bringing an old filesystem under database control or when rescuing a project after a manual move that was not done through the storage commands.

Note: Discovery will only work for paths/roots with the intended PhyloODB file system. You cannot point discovery to a folder of BUSCO results and expect the program to understand. The structure must be /ROOT/ACCESSION/LIBRARY_results_

12.9 Rediscovery versus verify

These are related but not identical.

- `discover` is about finding what exists on disk and registering or rebinding it inside registered genomes roots.
- `verify-*` tasks are about reconciling and repairing specific classes of data already expected by the project.

`discover` already ingests BUSCO runs found under accession folders.

`verify-busco` is still the correct follow-up because it repairs metadata, artifacts, primary-run assignments, duplicates, and stale lineage state.

`verify-assembly --discover` is narrower than `discover`:

- `discover` is filesystem-first and can register or rebind accessions from a chosen root/path;
- `verify-assembly --discover` is DB-first and helps re-mark known accessions from their current bound folders.

Typical examples:

```
phyloODB project.db queue verify-assembly --accessions @MOVED_SET --
repair --tidy
phyloODB project.db queue verify-busco --accessions @MOVED_SET --repair
phyloODB project.db queue verify-libraries --library-name metazoa_core --
repair
```

A sensible recovery pattern is often:

1. use `discover --dry-run`;
2. rerun with `--overwrite` if needed;
3. run the relevant verify task with `--repair`;
4. inspect reports before resuming larger workflows.

For recovery of an old PhyloODB filesystem on a different drive:

1. register the old genomes location as a genomes root if needed;
2. activate it only if you want new downloads to land there;
3. run root-scoped discovery;
4. run verification on the same root.

Example:

```
phyloODB project.db storage add-root --kind genomes --base-path
/mnt/hdd/old_project/genomes --label HDD_GENOMES
phyloODB project.db discover --root HDD_GENOMES
phyloODB project.db queue verify-assembly --root HDD_GENOMES --repair --
tidy
phyloODB project.db queue verify-busco --root HDD_GENOMES --repair
phyloODB project.db queue verify-libraries --repair
phyloODB project.db queue verify-orthofinder --repair
```

12.10 Recommended operator discipline

The following habits prevent most storage and recovery problems.

1. Prefer `storage move-genomes` and `storage move-libraries` over manual filesystem moves.
2. If you do move data manually, follow with `storage ... --rebind-only` or `discover --overwrite`.
3. Rebind whole roots only when the entire subtree has moved intact.
4. Keep root paths broad and intentional. Do not let operational roots fragment into many one-off directories.
5. Treat `purge` as a surgical cleanup tool, not a routine reset mechanism.
6. Use verify tasks after structural changes.
7. Activate a new working root explicitly. Do not assume adding a root changes where new work will go.

With those rules, a long-lived project can evolve across drives and over time without losing track of what its files mean.

13. Practical advice

- Use `list assemblies` extensively before downloading large sets.
- Store important accession panels as variables rather than relying on shell history.
- Prefer explicit task names and `queue` in long-running shared projects, even though `run` now follows a single task chain to completion in its own temporary daemon.
- Keep derived libraries separate in concept from target datasets. A reference panel and a target panel often serve different roles.
- Treat decontamination and paralog filtering as iterative quality screens. They are most useful when their results are inspected and, if necessary, rerun with revised settings.

14. Further reading in this repository

This manual is the primary user-facing document. The following companion files are intended to be used alongside it.

- [docs/QUICKSTART.md](#) for a first-run handbook with the shortest path from empty database to basic export.
- [docs/COMMAND_REFERENCE.md](#) for the command tree and concise task summaries.
- [tutorial/README.md](#) for a worked mammal tutorial centred on chimpanzee and broader primate sampling.
- [tutorial/chimp_mammal_core_example.sh](#) for a commented shell example showing how the tutorial can be translated into a queue-driven command script.

15. References

PhyloODB coordinates several external databases and command-line tools. Cite the specific versions, lineage datasets, and accession records used in a study; the references below are a starting point for common PhyloODB workflows.

Core resources:

- NCBI Taxonomy: Federhen, S. (2012). [The NCBI Taxonomy database](#). *Nucleic Acids*

Research, 40(D1), D136-D143.

- GenBank: cite the current GenBank update paper and the exact accession.version identifiers used in the study. See the [NCBI GenBank overview](#).
- RefSeq: cite the appropriate RefSeq publication for the release or resource used. See [Citing RefSeq](#).
- OrthoDB/BUSCO datasets: Tegenfeldt, F. et al. (2024). [OrthoDB and BUSCO update: annotation of orthologs with wider sampling of genomes](#). *Nucleic Acids Research*.

Core analysis software:

- BUSCO: Manni, M. et al. (2021). [BUSCO Update: Novel and Streamlined Workflows along with Broader and Deeper Phylogenetic Coverage](#). *Molecular Biology and Evolution*, 38(10), 4647-4654.
- OrthoFinder: Emms, D. M. and Kelly, S. (2019). [OrthoFinder: phylogenetic orthology inference for comparative genomics](#). *Genome Biology*, 20, 238.
- MAFFT: Katoh, K. and Standley, D. M. (2013). [MAFFT Multiple Sequence Alignment Software Version 7](#). *Molecular Biology and Evolution*, 30(4), 772-780.
- IQ-TREE 2: Minh, B. Q. et al. (2020). [IQ-TREE 2: New Models and Efficient Methods for Phylogenetic Inference in the Genomic Era](#). *Molecular Biology and Evolution*, 37(5), 1530-1534.
- FastTree 2: Price, M. N., Dehal, P. S. and Arkin, A. P. (2010). [FastTree 2: Approximately Maximum-Likelihood Trees for Large Alignments](#). *PLOS ONE*, 5(3), e9490.
- Siu-Ting, K., Torres-Sánchez, M., San Mauro, D., Wilcockson, D., Wilkinson, M., Pisani, D., O'Connell, M.J., Creevey, C.J., 2019a. Inadvertent Paralog Inclusion Drives Artifactual Topologies and Timetree Estimates in Phylogenomics. *Mol. Biol. Evol.* 36, 1344–1356. <https://doi.org/10.1093/molbev/msz067>
- Pisani, D., Rossi, M.E., Marlétaz, F., Feuda, R., 2022. Phylogenomics: Is less more when using large-scale datasets? *Curr. Biol.* 32, R1340–R1342. <https://doi.org/10.1016/j.cub.2022.11.019>

Search, clustering, and gene-prediction tools:

- NCBI BLAST+: Camacho, C. et al. (2009). [BLAST+: architecture and applications](#). *BMC Bioinformatics*, 10, 421.
- DIAMOND: Buchfink, B., Xie, C. and Huson, D. H. (2015). [Fast and sensitive protein](#)

[alignment using DIAMOND](#). *Nature Methods*, 12, 59-60.

- CD-HIT: Li, W. and Godzik, A. (2006). [Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences](#). *Bioinformatics*, 22(13), 1658-1659.
- Miniprot: Li, H. (2023). [Protein-to-genome alignment with miniprot](#). *Bioinformatics*, 39(1), btad014.
- MetaEuk: Levy Karin, E., Mirdita, M. and Soeding, J. (2020). [MetaEuk: sensitive, high-throughput gene discovery, and annotation for large-scale eukaryotic metagenomics](#). *Microbiome*, 8, 48.
- AUGUSTUS: Stanke, M. and Morgenstern, B. (2005). [AUGUSTUS: a web server for gene prediction in eukaryotes that allows user-defined constraints](#). *Nucleic Acids Research*, 33(Web Server issue), W465-W467.